

“AUTOMATIC WATERING SYSTEM FOR PLANTS”

A project report submitted in partial fulfillment of requirements for the award of degree of

BACHELOR OF COMPUTER APPLICATIONS



Submitted By:

Miss. Bhavana Shahapurkar

USN:U15BO23S0039

Miss. Netra Belgundkar

USN:U15BO23S0046

Miss. Samiksha Babugawade

USN:U15BO23S0062

Under the Guidance of:

Prof. Smita Desai

BHARTESH COLLEGE OF COMPUTER APPLICATIONS

Affiliated to Rani Channamma University, Belagavi

2025-2026

Bharatesh Education Trust's

BHARATESH COLLEGE OF COMPUTER APPLICATIONS

B.C.188, Dharwad Road, Belgavi-590 016



Certificate

This is to certify that the project entitled **“Automatic watering system for plants”** was satisfactorily completed by **Ms. Bhavana Shahapurkar. (U15BO23S0039)**, **Ms. Netra Belgundkar. (U15BO23S0046)**, **Ms. Samiksha Babugawade. (U15BO23S0062)** under the partial fulfillment of requirement of the degree of Bachelor of Computer Applications under Rani Channamma University, Belagavi from February 2025 to May 2026

Internal Guide

Prof. Smita Desai

Principal

Dr. Mahesh Mardolkar

Examiners:

1. _____

2. _____

ACKNOWLEDGEMENT

It is a great pleasure to acknowledge all those who have contributed towards the conception, origin and nurturing of this project "Cyber security threat detection". We would like to extend thanks to many people, who so generously contributed to the work presented in this report and like to place on record our heartfelt thanks to each one of them who helped us in this project whichever and whenever needed.

We are grateful our enthusiastic principal, **Dr. Mahesh Mardolkar**, for active interest he showed in our project and similar, profound gratitude goes to our internal guide **Prof. Smita Desai** for guiding us in the project with her suggestions, co-operation and encouragement throughout our project.

Special mention goes to all the faculties of BCCA, our friends for giving immense support and motivation. Finally, thanks go to our parents for their support and yes, the almighty God abetted us a lot to complete this project successfully.

With Heartfelt Thanks to One and All

Ms. Bhavana Shahapurkar

Ms. Netra Belgundkar

Ms. Samiksha Babugawade

SYNOPSIS

ABSTRACT

The Smart Automatic Plant Watering System is an IoT-based solution designed to automate plant irrigation, send real-time mobile notifications, and store data in a database for monitoring and analysis. This system uses a soil moisture sensor to detect the dryness of the soil and automatically activates a water pump when required. Whenever watering starts, a notification is sent to the user's mobile device.

Additionally, all sensor data such as soil moisture, temperature, humidity, and watering status are stored in a database. This allows users to track plant conditions over time and analyze patterns. The system is built using NodeMCU (ESP8266), making it efficient, smart, and suitable for modern agriculture and home gardening.

INTRODUCTION

With the integration of IoT and data storage technologies, plant watering systems can now be automated and monitored efficiently. This project introduces a smart irrigation system that not only waters plants automatically but also sends mobile notifications and stores data in a database. The stored data helps users understand plant conditions and improve watering strategies. This system is useful for smart homes, gardens, and agriculture.

EXISTING SYSTEM

The existing system relies entirely on manual watering, where plants are cared for by hand without any form of automation. There are no sensors, timers, or smart controls in place to regulate watering or monitor environmental conditions. Additionally, the system does not provide any mobile notifications, so users are not alerted about the status or needs of their plants. It also lacks data storage, meaning no historical information about watering patterns or plant conditions is recorded. As a result, there is no analysis or insight into plant health, making the process time-consuming, less efficient, and dependent solely on human judgment.

PROPOSED SYSTEM

The proposed system automates plant watering, sends real-time notifications, and stores data in a database for analysis. This system allows users to monitor plant health over time and make better decisions based on stored data. It combines IoT and database technology for smart plant management.

SYSTEM SPECIFICATION

Software Requirements:-

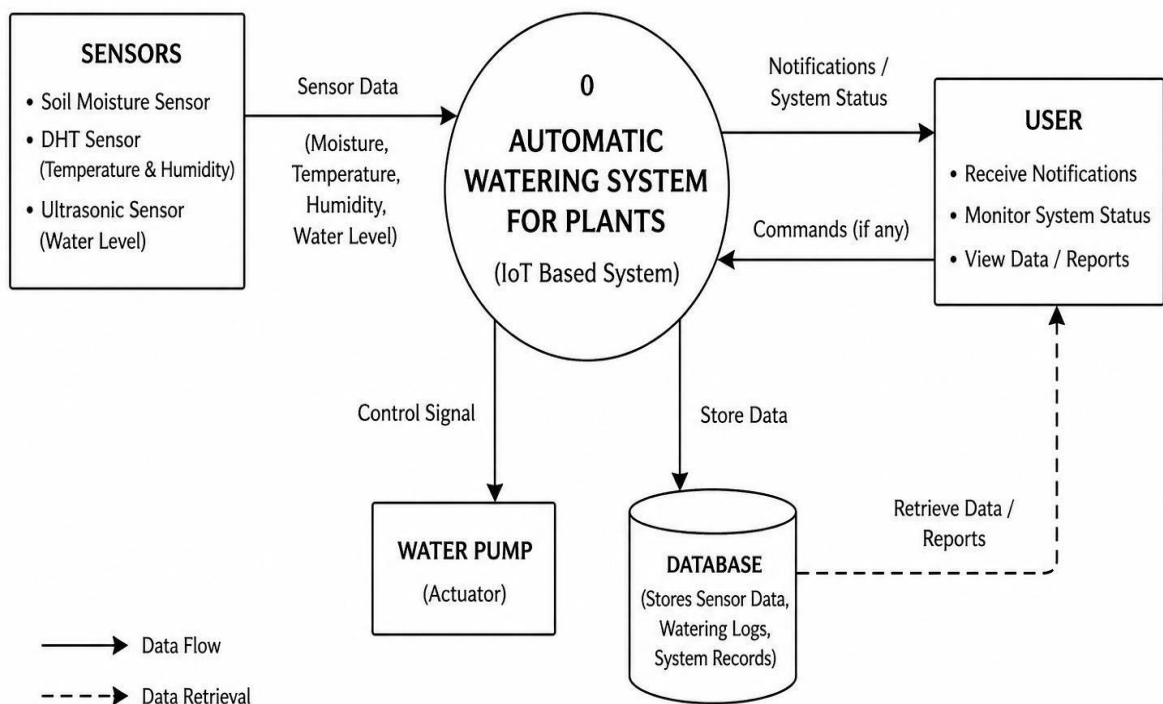
- **Arduino IDE:** Used to write and upload code to NodeMCU.
- **Embedded C/C++:** Programming language for system logic.
- **Firmware (ESP8266):** Controls sensors and pump operation.
- **IoT Platform (ThingSpeak/Blynk):** Used for remote monitoring.
- **Database:** Stores sensor data and watering records.
- **Mobile/Web App:** Used for notifications and monitoring.

Hardware Requirements:-

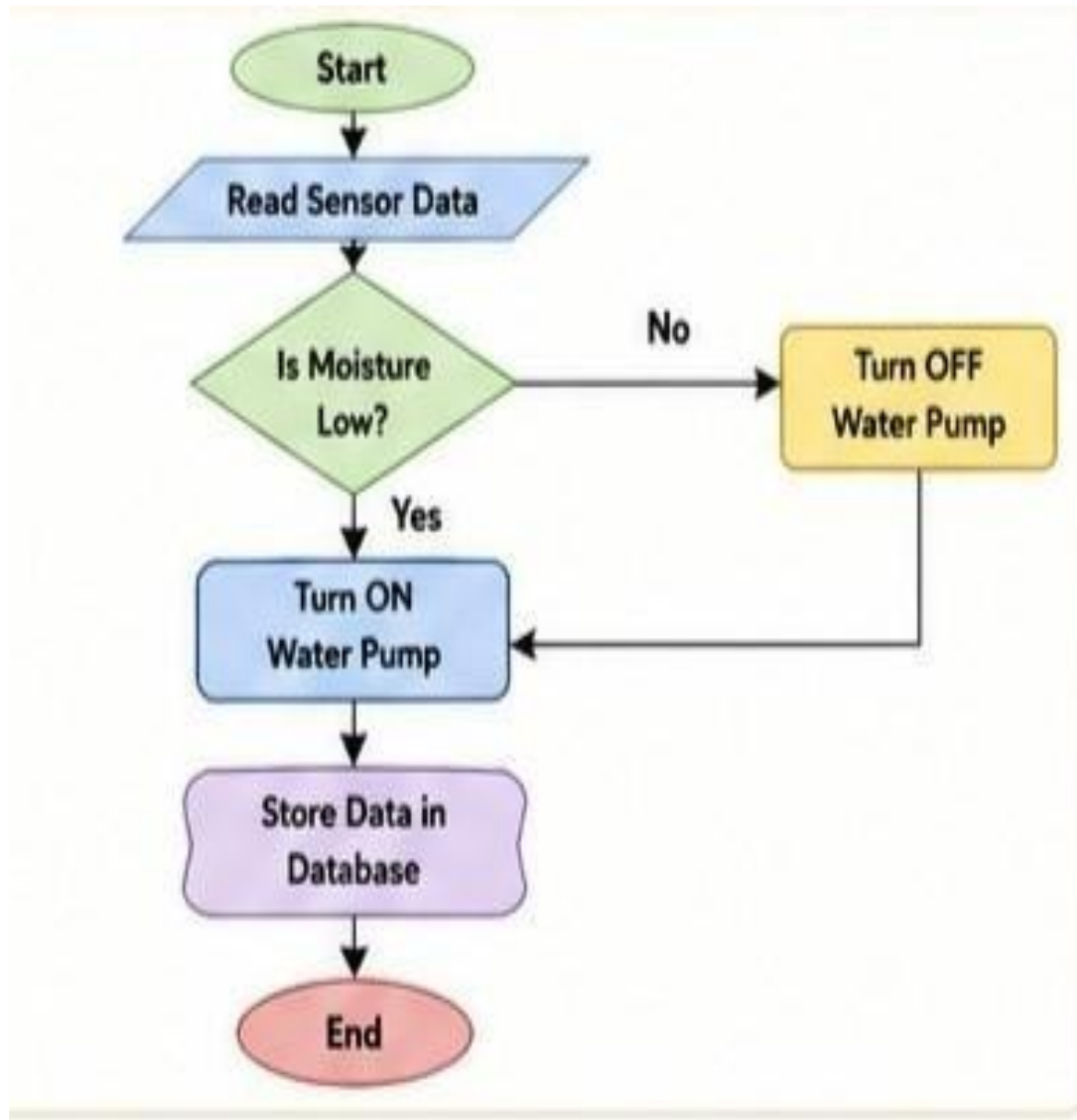
- **Node MCU (ESP8266):** Main controller of the system.
- **Power Supply:** Provides power to the system.
- **Soil Moisture Sensor:** Detects soil dryness.
- **DHT Sensor:** Measures temperature and humidity.
- **Relay Module:** Controls the water pump.
- **Water Pump:** Supplies water to plants.
- **Ultrasonic Sensor:** Detects water level.
- **Buzzer:** Gives alert notifications.
- **P4 Connector & Jumper Wires:** Used for connections.

Data flow diagram 0 level

DATA FLOW DIAGRAM – LEVEL 0 (Context Diagram)



Flow chart



CONCLUSION

The Automatic Watering System for Plants is an efficient and smart solution for modern plant care. It automates the watering process based on soil moisture levels, reducing manual effort and saving water. The system also provides real-time monitoring and notifications, helping users take better care of their plants.

By integrating IoT technology and data storage, the system improves accuracy and reliability in plant maintenance. Overall, it is a cost-effective and useful system for homes, gardens, and agriculture, ensuring healthy plant growth and efficient resource usage.

INDEX

	CONTENT	<u>Page no</u>
	INTRODUCTION	<u>17-18</u>
	OBJECTIVE AND SCOPE OF SYSTEM	<u>19-21</u>
	THEORETICAL BACKGROUND AND DEFINITION OF PROBLEM 3.1 Existing System 3.2 Proposed System	<u>22-25</u>
	SYSTEM ANALYSIS AND DESIGN 4.1 System analysis 4.2 Feasibility Study 4.3 System design	<u>26-31</u>
	USER REQUIREMENT AND SYSTEM PLANNING 5.1 System Requirement 5.2 Functional Requirement 5.3 Non Functional Requirement	<u>32-36</u>

	METHODOLOGY ADOPTED	<u>37-41</u>
	6.1 Gantt Chart	
	6.2 Software Process	
	6.3 Spiral Model	

	SYSTEM IMPLEMENTATION AND DETAILS HARDWARE AND SOFTWARE	<u>42-46</u>
	7.1 Hardware Requirements	
	7.2 Software Requirements	
	SYSTEM MAINTENANCE AND EVALUATION	<u>40-43</u>
	8.1 System Maintenance	
	8.2 System Evaluation	
	COST AND BENEFIT ANALYSIS	<u>44-46</u>
	9.1 Costs	
	9.2 Benefits	
	DETAILED LIFE CYCLE OF THE PROJECT	<u>47-52</u>
	10.1 Life Cycle Phases	
	10.2 Dataflow Diagram	
	SNAPSHOTS	<u>53-56</u>
	SYSTEM TESTING	<u>57-58</u>

	SOURCE CODE	<u>59-73</u>
	13.1 project.ino	
	13.2 Modules.py	
	13.3 Base.html	
	13.4 About.html	

	ANNEXURE	<u>74-75</u>
	14.1 Data Dictionary	
	14.2 Importance Of Data Dictionary	
	14.3 Abbreviations	
	14.4 Figures	
	CONCLUSION	<u>76</u>
	FUTURE ENHANCEMENTS	<u>77-78</u>
	REFERENCES	<u>79</u>

INTRODUCTION

1.INTRODUCTION

In today's world, technology is playing a major role in improving efficiency and reducing manual effort in various fields, including agriculture and home gardening. Proper watering is one of the most important factors for healthy plant growth. However, traditional methods of watering plants depend completely on human effort and judgment.

This often leads to problems such as overwatering, underwatering, and irregular maintenance, which can negatively affect plant health and lead to wastage of water. With the advancement of the Internet of Things (IoT), it has become possible to automate many daily tasks, including plant care.

IoT allows devices to communicate with each other and share data over the internet, making systems smarter and more efficient. By integrating sensors, microcontrollers, and communication technologies, plant watering can be automated and monitored in real time. The Automatic Watering System for Plants is an IoT-based solution designed to address these issues. This system uses a soil moisture sensor to measure the moisture content of the soil.

When the soil becomes dry and falls below a certain threshold, the system automatically activates a water pump to irrigate the plant. Once the required moisture level is reached, the pump is turned off, ensuring optimal water usage. In addition to soil moisture sensing, the system also includes a DHT sensor to monitor environmental conditions such as temperature and humidity.

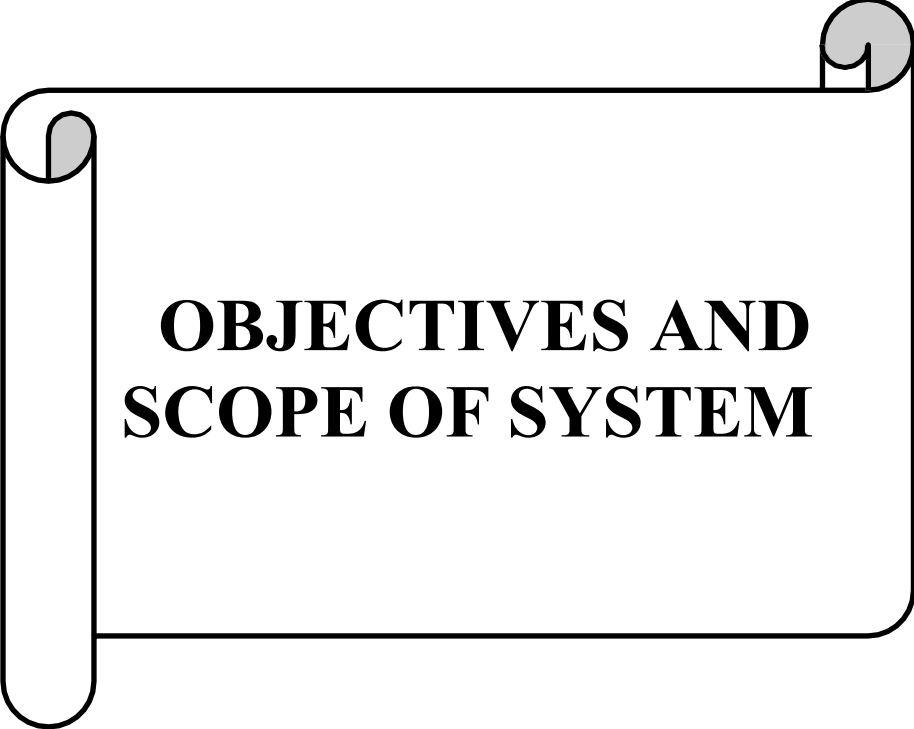
These factors play an important role in plant growth and help in understanding the surrounding environment. An ultrasonic sensor is used to detect the water level in the storage tank, ensuring that the system operates only when sufficient water is available.

The system is built using NodeMCU (ESP8266), which is a low-cost microcontroller with built-in Wi-Fi capability. This allows the system to connect to the internet and provide

remote access to users. Through IoT platforms or mobile applications, users can monitor real-time data, receive notifications when watering starts, and track plant conditions from anywhere.

Another important feature of the system is data storage. All sensor readings, watering activities, and system status are stored in a database. This stored data can be analyzed to understand patterns in plant care, improve watering strategies, and make better decisions in the future. It also helps in maintaining a record of plant health over time. The proposed system is highly useful for various applications such as home gardening, smart homes, greenhouses, and agriculture.

It reduces the need for constant human supervision, saves water by providing only the required amount, and ensures that plants receive proper care even in the absence of the user. Overall, the Automatic Watering System combines automation, IoT, and data analysis to create an intelligent and efficient solution for plant maintenance. It not only improves plant growth but also promotes sustainable use of water resources, making it an important step towards smart agriculture and modern living.



OBJECTIVES AND SCOPE OF SYSTEM

2.OBJECTIVES AND SCOPE OF SYSTEM

❖ OBJECTIVES OF THE SYSTEM

1.To automate the irrigation process

The system is designed to reduce manual effort by automatically controlling the watering of plants. It continuously monitors soil conditions and activates the water pump only when required. This ensures that plants receive water without human intervention.

2.To monitor soil moisture levels in real time

The system uses a soil moisture sensor to measure the water content present in the soil. This real-time monitoring helps the system understand the current condition of the soil and take appropriate actions based on accurate data.

3.To optimize water usage and reduce wastage

One of the main objectives of the system is to use water efficiently. By watering plants only when necessary, the system avoids overwatering and minimizes water wastage, which is especially important in areas facing water scarcity.

4.To implement machine learning for intelligent decision-making

The system uses a trained machine learning model to analyze data and predict irrigation needs. Instead of relying only on fixed threshold values, the model improves decision accuracy by considering patterns in the dataset.

5.To provide a user-friendly monitoring interface

A web-based application developed using Django allows users to monitor system activity. Users can view soil moisture data, irrigation status, and prediction results through an easy-to-use dashboard.

6.To store and manage irrigation data for analysis

The system stores sensor readings and irrigation records in a database. This data can be used for future analysis, helping users understand patterns and improve irrigation planning.

7.To improve plant health and growth

By maintaining proper soil moisture levels, the system ensures that plants receive the right amount of water. This leads to better growth, improved yield, and healthier plants.

8.To reduce human errors in irrigation

Manual watering often leads to mistakes such as irregular watering or incorrect quantity. The automated system eliminates these errors by making consistent and data-driven decisions.

9.To create a scalable and extendable system

The system is designed in such a way that it can be expanded in the future. Features like IoT integration, mobile control, and weather-based irrigation can be added easily.

❖ SCOPE OF THE SYSTEM

- **Real-time monitoring of environmental conditions**

The system continuously collects data from soil moisture sensors. This allows it to operate based on current soil conditions rather than assumptions, ensuring accurate irrigation control.

- **Automated control of water pump**

The system controls the water pump automatically based on the processed data. When the soil is dry, the pump is turned on, and when sufficient moisture is detected, it is turned off.

- **Integration of machine learning model for prediction**

The system includes a trained machine learning model that analyzes input data and predicts irrigation requirements. This makes the system more intelligent compared to basic automation systems.

- **Web-based monitoring and control using Django**

The scope includes a web application where users can log in and monitor system data. The interface displays sensor readings, irrigation status, and predictions in a clear and organized manner.

- **Data storage and management**

All sensor data and irrigation activities are stored in a database. This allows users to track historical data and analyze trends over time.

- **Application in different environments**

The system can be used in home gardens, agricultural fields, greenhouses, and research setups. It is suitable for both small-scale and medium-scale irrigation needs.

- **Support for decision-making through data analysis**

By providing stored data and prediction results, the system helps users make better decisions regarding irrigation planning and water usage.

- **Dependence on hardware and environmental conditions**

The system operates based on sensor input and requires proper installation. Its performance depends on sensor accuracy, power supply, and environmental factors such as soil type.

- **Provision for future enhancements**

The current system provides a foundation that can be extended with advanced features such as IoT-based remote monitoring, mobile applications, weather integration, and AI-based optimization.

- **Limitations within the current implementation**

The system currently depends on predefined datasets and sensor readings. While it provides intelligent predictions, its accuracy can improve further with more data and advanced models.

.

**THEREOTICAL
BACKGROUND AND
DEFINITION OF PROBLEM**

3.THEORETICAL BACKGROUND AND DEFINITION OF PROBLEM

3.1 EXISTING SYSTEM

The existing irrigation systems are mostly traditional and lack intelligence. They are either manual or semi-automatic and do not use real-time data for decision-making.

- **Manual Irrigation System**

In this method, watering is done by humans using buckets, pipes, or drip systems. The decision of when and how much to water depends entirely on the user's observation. This method is time-consuming and often leads to errors such as overwatering or underwatering.

- **Timer-Based Irrigation System**

Some systems use timers to automate watering at fixed intervals. Although this reduces manual effort, it does not consider the actual soil condition. Water is supplied even when the soil already has sufficient moisture, which leads to wastage.

- **Lack of Real-Time Monitoring**

Existing systems do not continuously monitor soil moisture levels. They operate without actual data, making them inefficient and unreliable for proper plant care.

- **No Intelligent Decision-Making**

There is no use of machine learning or data analysis in traditional systems. Decisions are either manual or based on fixed rules, which cannot adapt to changing environmental conditions.

- **No Data Storage or Analysis**

Traditional systems do not store irrigation data. Users cannot track past watering patterns or analyze data to improve irrigation practices.

- **No User Interface or Remote Access**

Most existing systems do not provide a dashboard or application for monitoring. Users cannot check system status or control irrigation remotely.

- **High Water Wastage and Inefficiency**

Due to lack of proper monitoring and intelligence, water is often used unnecessarily. This leads to wastage of water and affects sustainability.

3.2 PROPOSED SYSTEM

The proposed system introduces automation, intelligence, and real-time monitoring to improve irrigation efficiency. It integrates sensors, machine learning, and a web-based application.

•Sensor-Based Soil Moisture Monitoring

The system uses a soil moisture sensor to measure the water content in the soil continuously. This real-time data helps the system understand the exact condition of the soil.

•Automated Irrigation Control

Based on sensor data and machine learning predictions, the system automatically controls the water pump. It turns the pump ON when the soil is dry and OFF when sufficient moisture is reached.

•Machine Learning-Based Prediction

A trained machine learning model is used to analyze irrigation data and predict watering needs. This improves decision-making compared to simple threshold-based systems.

•Web-Based Monitoring using Django

The system includes a Django web application that provides a user-friendly dashboard. Users can log in, view sensor data, check irrigation status, and see prediction results.

•Data Storage and Management

All sensor readings and irrigation activities are stored in a database. This allows users to track historical data and analyze patterns over time.

•Real-Time Data Visualization

The dashboard displays data in graphical and tabular formats. This helps users understand system performance easily.

•Reduced Water Wastage

By watering plants only when necessary, the system ensures efficient use of water and supports sustainable practices.

•Improved Plant Health

Maintaining proper soil moisture levels ensures better plant growth and reduces the risk of damage caused by improper watering.

•Scalable and Extendable Design

The system can be enhanced in the future with features such as IoT integration, mobile applications, and weather-based irrigation control.

❖ SYSTEM ARCHITECTURE

The system architecture defines how different components of the irrigation system interact with each other. It consists of hardware components, software modules, and data flow mechanisms.

- **Data Collection Layer**

This layer includes the soil moisture sensor, which collects real-time data from the soil. The sensor measures moisture levels and sends the data to the processing unit.

- **Data Processing Layer**

In this layer, the collected data is processed by the backend system developed using Django. The system cleans and prepares the data for further analysis.

- **Machine Learning Layer**

The processed data is passed to the trained machine learning model. The model analyzes the data and predicts whether irrigation is required based on learned patterns.

- **Control Layer**

Based on the prediction results, the system sends signals to control the water pump through a relay module. The pump is activated or deactivated automatically.

- **Database Layer**

All data, including sensor readings, user information, and prediction results, is stored in a database. This ensures proper data management and supports analysis.

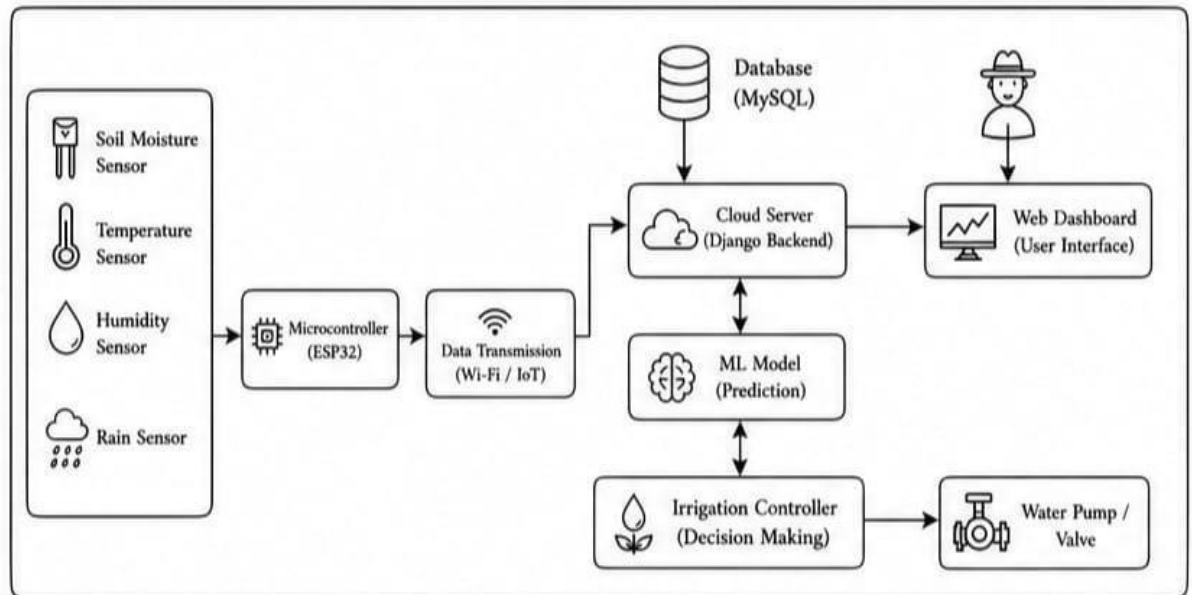
- **User Interface Layer**

The Django web application acts as the user interface. It allows users to log in, view data, monitor system status, and interact with the system through a dashboard.

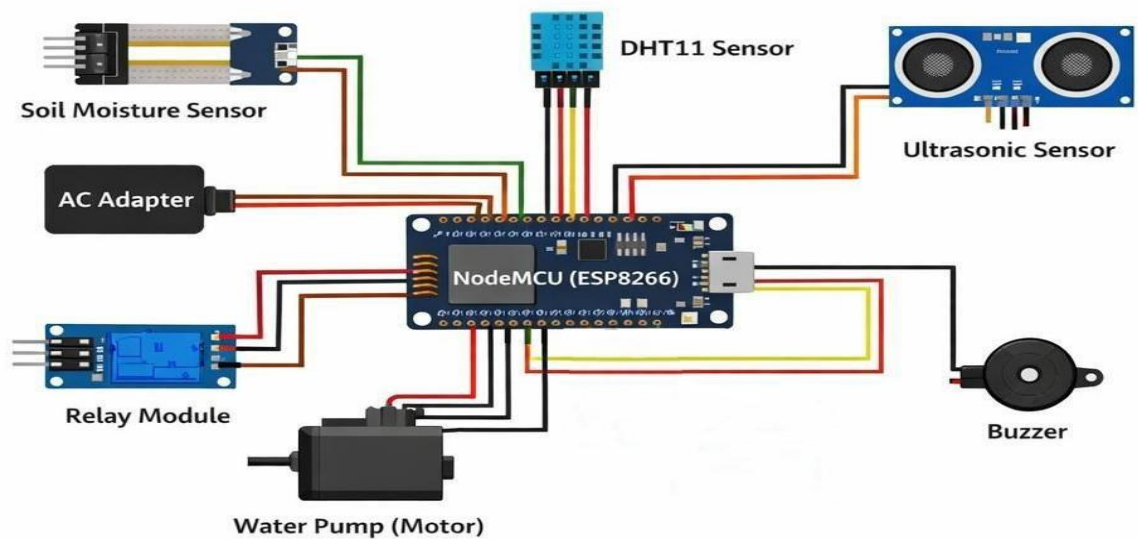
- **Data Flow Mechanism**

The overall flow of the system starts from data collection through sensors, followed by processing, prediction using machine learning, decision-making, and finally displaying the results on the user interface.

- **SYSTEM ARCHTECTURE DESIGN**



- **Circuit Diagram**



**SYSTEM ANALYSIS AND
DESIGN**

4. SYSTEM ANALYSIS AND DESIGN

4.1 SYSTEM ANALYSIS

System analysis is an important phase in system development where the requirements of the system are studied and understood in detail. In the Smart Automatic Irrigation System, this phase focuses on identifying the problems in traditional irrigation methods and defining how the proposed system will solve them using modern technology.

The analysis includes studying how soil moisture data is collected, how decisions are made using machine learning, and how the system interacts with users through a web interface. It also considers the integration of hardware components like sensors and pumps with software components such as the Django backend and database.

➤ Key Points of System Analysis:

- **Requirement Identification:** The system requires real-time monitoring of soil moisture, automatic control of irrigation, and a user-friendly interface for monitoring and analysis.
- **Problem Understanding:** Traditional systems lack automation and intelligence, leading to water wastage and inefficient irrigation.
- **System Functionality Analysis:** The system must collect sensor data, process it, predict irrigation needs, and control the water pump accordingly.
- **Performance Consideration:** The system should respond quickly to changes in soil conditions and operate continuously without delays.
- **Reliability and Accuracy:** The system must provide accurate results based on sensor data and machine learning predictions.
- **Constraints Analysis:** The system depends on hardware components, power supply, and environmental conditions such as soil type.

4.2 FEASIBILITY STUDY

Feasibility study evaluates whether the proposed system can be successfully implemented in practical conditions. It examines different aspects to ensure that the system is viable and beneficial.

1. Technical Feasibility

The system is technically feasible because it uses commonly available technologies such as soil moisture sensors, Python programming, Django framework, and machine learning models. These technologies are well-supported and can be easily integrated to develop the system.

2. Economic Feasibility

The overall cost of the system is low as it uses affordable hardware components and open-source software tools. The system helps save water and reduces labor costs, making it economically beneficial in the long run.

3. Operational Feasibility

The system is simple to operate and does not require advanced technical knowledge. Users can easily monitor the system through a web interface, making it suitable for farmers and home users.

4. Social Feasibility

The system promotes efficient use of water resources and supports sustainable agricultural practices. It benefits society by reducing water wastage and improving productivity.

5. Environmental Feasibility

The system helps in conserving water and maintaining ecological balance by preventing unnecessary irrigation.

4.3 SYSTEM DESIGN

System design defines how the system will be structured and how different components will work together. The design of the Smart Automatic Irrigation System ensures proper integration of hardware and software components.

1. Architectural Design

The system follows a layered structure where data is collected from sensors, processed in the backend, analyzed using a machine learning model, and then used to control the water pump. The results are displayed through a web interface.

2. Data Flow Design

The data flows from the soil moisture sensor to the backend system. It is then processed and analyzed by the machine learning model. Based on the output, the system controls irrigation and stores data in the database for future use.

3. Module Design

The system is divided into modules such as sensor module, backend processing module, machine learning module, database module, and user interface module. Each module performs a specific function and contributes to the overall system.

4. Database Design

The database is used to store user details, sensor data, and prediction results. It ensures proper management and retrieval of information.

5. Interface Design

The system provides a web-based dashboard where users can view real-time data and system status. The interface is designed to be simple and user-friendly.

6. Control Design

The control system manages the operation of the water pump. It activates or deactivates the pump based on sensor readings and machine learning predictions.

**USER REQUIREMENT
AND SYSTEM PLANNING**

5. USER REQUIREMENT AND SYSTEM PLANNING

5.1 SYSTEM REQUIREMENTS

System requirements describe the overall needs of the system, including both hardware and software components required for proper functioning.

Hardware Requirements

- **Soil moisture sensor:**

This sensor is required to measure the moisture level of the soil. It provides real-time data which is essential for making irrigation decisions.

- **Microcontroller (Arduino or similar device):**

It is used to read data from the sensor and communicate with the system for processing and control.

- **Relay Module:**

The relay acts as a switch to control the water pump based on signals received from the system.

- **Water Pump:**

The pump is responsible for supplying water to plants when irrigation is required.

- **Power Supply:**

A stable power source is necessary to ensure continuous operation of the system.

Software Requirements

- **Python Programming Language:**

Used for backend development and implementing machine learning logic.

- **Django Framework**

Provides the structure for developing the web-based application and managing user interactions.

- **Machine Learning Libraries (such as Scikit-learn):**

Used to train and implement the model for predicting irrigation needs.

- **Database (SQLite or similar):**

Stores user data, sensor readings, and prediction results.

- **Frontend Technologies (HTML, CSS, JavaScript):**

Used to design the user interface and dashboard for displaying system data.

5.2. FUNCTIONAL REQUIREMENTS

Functional requirements define what the system should do. These describe the core operations and features of the system.

- **User Registration and Login:**

The system should allow users to create an account and log in securely to access the dashboard.

- **Soil Moisture Data Collection:**

The system should collect real-time soil moisture data from the sensor.

- **Data Processing and Analysis:**

The system should process the collected data and prepare it for prediction.

- **Machine Learning Prediction:**

The system should use a trained machine learning model to predict whether irrigation is required.

- **Automatic Irrigation Control:**

The system should automatically turn the water pump ON or OFF based on prediction results.

- **Dashboard Display:**

The system should display real-time data, irrigation status, and prediction results on a web-based dashboard.

- **Data Storage:**

The system should store sensor readings and irrigation history in a database.

- **Alert or Notification System:**

The system should notify users when irrigation is activated or when soil moisture is low.

- **User Interaction with System:**

The system should allow users to view data and monitor system performance through the interface.

5.3. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define how the system performs and its quality attributes.

- **Performance**

The system should process sensor data and provide results quickly without delay.

- **Reliability**

The system should operate continuously and provide accurate results based on sensor data.

- **Usability**

The system should have a simple and user-friendly interface that can be used easily by non-technical users.

- **Scalability**

The system should be designed in a way that allows future enhancements such as IoT integration and mobile applications.

- **Security**

The system should ensure secure user authentication and protect stored data.

- **Maintainability**

The system should be easy to maintain and update when required.

- **Efficiency**

The system should optimize water usage and reduce unnecessary resource consumption.

- **Availability**

The system should be available for use at all times with minimal downtime.

- **Accuracy**

The system should provide precise results based on sensor input and machine learning predictions.

**METHODOLOGY
ADOPTED**

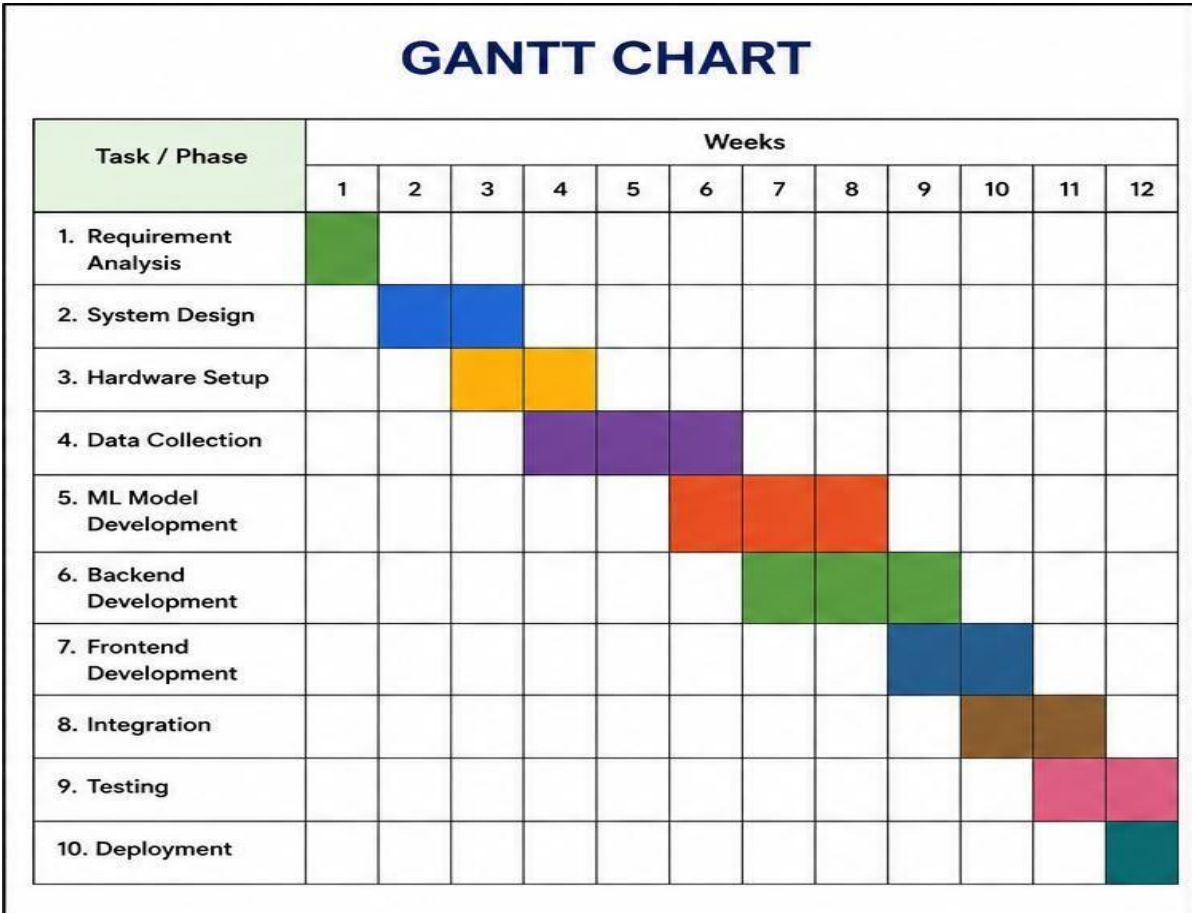
6. METHODOLOGY ADOPTED

The development of the Smart Automatic Irrigation System follows a structured and systematic approach to ensure that both hardware and software components work together efficiently. The methodology adopted in this project focuses on integrating sensor-based data collection, machine learning-based decision-making, and a web-based monitoring system developed using Django.

Initially, the requirements of the system were identified, including real-time soil moisture monitoring, automatic irrigation control, and user interaction through a dashboard. Based on these requirements, the system was designed by dividing it into modules such as sensor module, backend processing, machine learning model, database management, and user interface.

The development process involved collecting data using sensors, preparing datasets, training a machine learning model, and integrating the model with the Django backend. The system was implemented step by step, where each module was developed, tested, and improved. The final system combines hardware and software to provide an intelligent irrigation solution.

6.1 Gantt Chart



6.2 SOFTWARE PROCESS

1. Project Planning and Initiation

- **Requirements Gathering**

- **User Requirement Analysis:**

- Identify user needs such as automatic irrigation, real-time monitoring, and alert notifications.

- **System Requirement Identification:**

- Define requirements for sensors (soil moisture, temperature, water level), NodeMCU, machine learning model, and web dashboard.

- **Existing System Study:**

- Analyze traditional irrigation methods to identify limitations such as water wastage and manual control.

- **Feasibility Study**

- **Technical Feasibility:**

- Evaluate the integration of IoT hardware (NodeMCU, sensors) with Django-based software and machine learning model.

- **Operational Feasibility:**

- Ensure the system can operate automatically with minimal human intervention and is easy for users to understand.

- **Economic Feasibility:**

- Analyze the cost of hardware components and development; use of open-source tools reduces overall cost.

- **Resource Feasibility:**

- Ensure availability of required hardware components, software tools, and development environment.

- **Project Planning**

- **Timeline Definition:**

- Divide the project into phases such as analysis, design, implementation, testing, and deployment.

- **Task Allocation:**

- Assign responsibilities for hardware setup, software development, ML model development, and testing.

- **Resource Allocation:**

- Allocate tools such as Python, Django framework, database system, and hardware components.

2. System Design

- **Architecture Design**

- High-Level Architecture:**

- The system consists of three main layers:**

- Hardware Layer(Sensors, Node MCU, Relay, Pump) ,
Processing Layer(Machine Learning Model)

Application Layer (Django Web Interface)

- **Component Interaction:**

Sensors collect data → NodeMCU sends data → ML model processes → Decision generated → Pump controlled → Data displayed on dashboard.

- **Data Flow Design**

- **Input Data:**

Soil moisture, temperature, humidity, and water level readings.

- **Processing Flow:**

Data is processed by the backend system and passed to the machine learning model.

- **Output Data:**

Irrigation decision (Pump ON/OFF) and status display on dashboard.

- **Database Design**

- **Data Storage:**

Store sensor readings, user details, irrigation logs, and alerts.

- **Data Organization:**

Structured tables such as User, SensorData, IrrigationLog, and Alert are maintained.

3. Implementation

- **Hardware Implementation**

- Connect soil moisture sensor, DHT11, and ultrasonic sensor to Node MCU.
- Integrate relay module with water pump.
- Ensure proper power supply and circuit connections

- **Software Implementation**

- **Backend Development:**

Develop system logic using Django framework.

- **Frontend Development:**

Design user interface using HTML, CSS, and JavaScript.

- **Machine Learning Integration:**

Train and integrate model to predict irrigation requirements.

- **Integration Process**

- Combine hardware and software modules.
- Ensure smooth communication between sensors, backend, and actuators.

4. Testing

- **Unit Testing**

Test individual components such as sensors, ML model, and web interface.

- **System Testing**

Verify complete system functionality including data flow and pump control.

➤ **Performance Testing**

Check response time and accuracy of irrigation decisions.

➤ **Error Handling**

Identify and fix issues related to sensor readings, communication, and system logic.

5. Deployment

➤ **System Installation**

Install hardware setup in real environment (field or plant area).

➤ **Software Deployment**

Deploy Django application and connect it with hardware system.

➤ **System Configuration**

Configure database, sensors, and communication modules.

6. Maintenance

➤ **System Monitoring**

Continuously monitor sensor data and system performance.

➤ **Software Updates**

Update system features and fix bugs regularly.

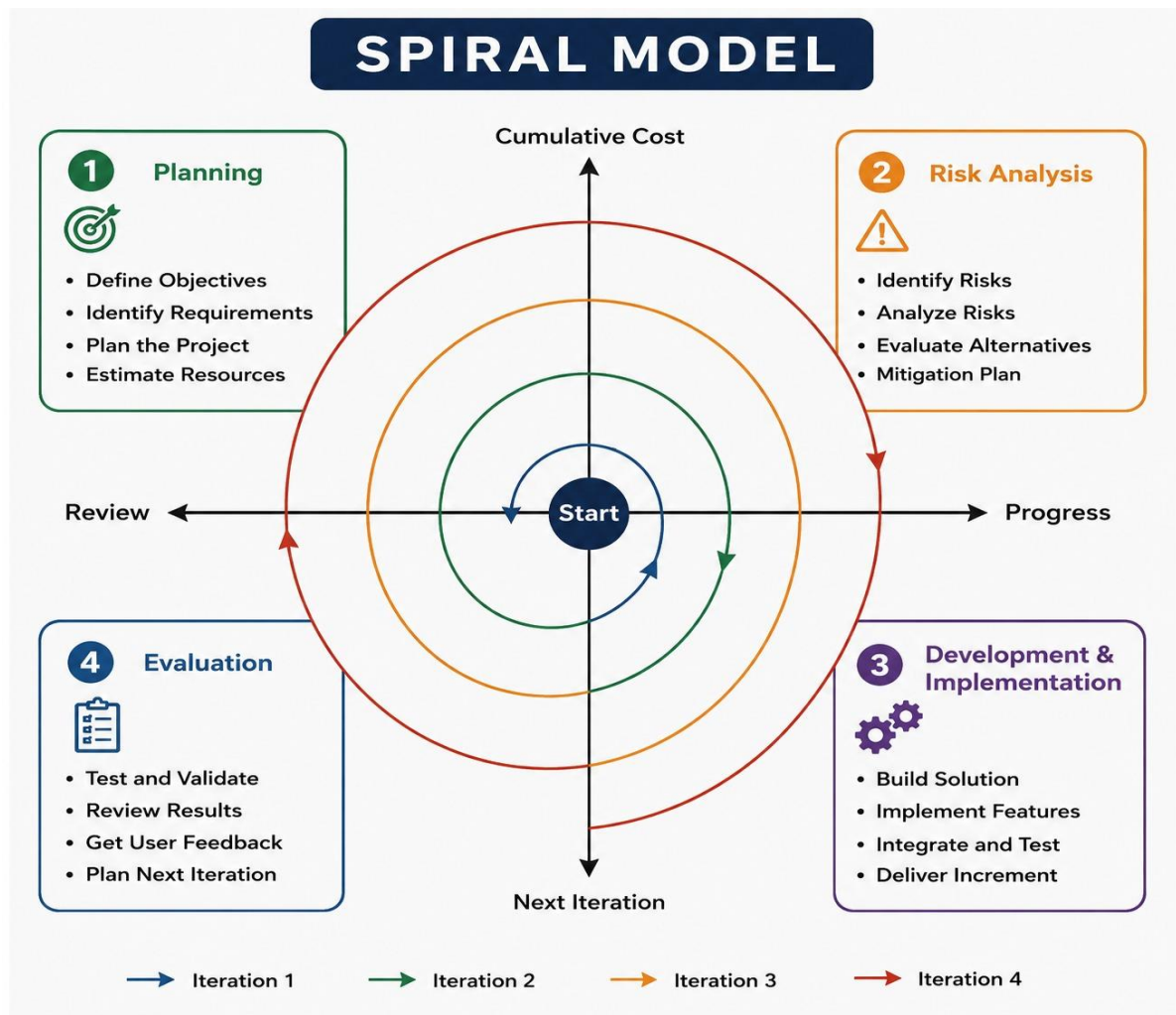
➤ **Model Improvement**

Retrain machine learning model using new data for better accuracy.

➤ **Hardware Maintenance**

Check sensors, relay, and pump for proper functioning.

6.2 SPIRAL MODEL



**SYSTEM IMPLEMENTATION
AND DETAILS OF
HARDWARE AND
SOFTWARE**

7.SYSTEM IMPLEMENTATION AND DETAILS OF HARDWARE AND SOFTWARE

The Smart Automatic Irrigation System is implemented by integrating hardware components with a software-based system. Soil moisture sensors collect real-time data, which is processed by a Django backend. A machine learning model analyzes the data and determines whether irrigation is required. Based on this, the system automatically controls the water pump using a relay module. The data and results are displayed on a web dashboard for user monitoring.

7.1. HARDWARE COMPONENTS

The Smart Automatic Irrigation System uses a set of hardware components to collect environmental data and control the irrigation process. All components are connected to the Node MCU, which acts as the central controller and manages the overall system operation.

- **Node MCU (ESP8266)**

Acts as the main controller of the system. It receives data from sensors, processes it, and sends signals to control devices like the relay and pump. It also supports communication with the software system.

- **Soil Moisture Sensor**

Measures the moisture level in the soil and provides input to the controller. This helps the system decide when irrigation is needed.

- **DHT11 Sensor**

Used to measure temperature and humidity. It provides additional environmental data for better monitoring of plant conditions.

- **4. Ultrasonic Sensor**

Measures the water level in the tank. It helps ensure that sufficient water is available before operating the pump.

- **Relay Module**

Acts as a switch to control the water pump. It receives signals from the NodeMCU and turns the pump ON or OFF.

- **Water Pump (Motor)**

Supplies water to the plants. It operates automatically based on system decisions.

- **Buzzer**

Provides alert signals for system status, such as low water level or pump activation.

- **Power Supply (AC Adapter)**

Supplies electrical power to all components for continuous operation.

7.2. SOFTWARE REQUIREMENTS

❖ Programming and Development

- **Python:**

Used for backend logic and machine learning implementation.

- **Django Framework:**

Used to develop the web application and manage system operations.

❖ Machine Learning and Data Processing

- **Machine Learning Libraries:**

Used to build and apply prediction models

- **Data Processing Libraries:**

Used to handle and prepare data for analysis.

❖ Database Management

- **SQLite / Relational Database:**

Stores system data such as sensor readings and results.

❖ Frontend Technologies

- **HTML, CSS, JavaScript:**

Used to design the dashboard and user interface.

❖ Development and Testing Tools

- **IDE (PyCharm / VS Code):**

Used for coding and debugging.

- **Version Control (Git):**

Used to manage code versions.

❖ **System Environment**

➤ **Operating System:**

Supports execution of the system.

➤ **Web Browser:**

Used to access and monitor the application.

**SYSTEM MAINTENANCE AND
EVALUATION**

8.SYSTEM MAINTENANCE AND EVALUATION

8.1System Maintenance

System maintenance is essential to ensure the continuous performance, reliability, and efficiency of the Smart Automatic Irrigation System. Since the system integrates hardware components, machine learning models, and a web-based application, regular monitoring and updates are required to maintain smooth operation. Maintenance activities help in preventing system failures, improving accuracy, and ensuring long-term usability.

➤ Maintenance Objectives

- **Ensure System Reliability:** Regular checking of sensors, NodeMCU, and relay connections is required to avoid system failures and ensure continuous irrigation control.
- **Improve System Accuracy:** Maintain accurate predictions by verifying sensor readings and updating the machine learning model when needed.
- **Optimize Performance:** Monitor system response time and ensure that data processing and irrigation control happen without delays.
- **Enhance Security:** Protect the web application by applying necessary updates and securing user data stored in the database.
- **Support System Improvement:** Identify issues and improve system functionality based on performance and user feedback.

➤ Maintenance Activities

❖ Regular Updates

- **Software Updates:**

The Django backend and frontend interface should be updated regularly to fix errors, improve features, and enhance performance.

- **Libraries and Frameworks:**

Python libraries and frameworks used for machine learning and development should be kept updated to ensure compatibility and security.

❖ Machine Learning Model Maintenance

- **Model Retraining:**

The machine learning model should be retrained periodically using updated data to improve prediction accuracy.

- **Performance Monitoring:**

The model's output should be checked regularly to ensure correct irrigation decisions. Hardware Maintenance

- **Sensor Checking:**

Soil moisture, temperature, and ultrasonic sensors should be checked and cleaned to ensure accurate readings.

- **Pump and Relay Inspection:**
The water pump and relay module should be tested regularly to ensure proper functioning.
- **Power Supply Verification:**
Ensure stable power supply to avoid interruptions in system operation.
- ❖ **Security Maintenance**
 - **User Authentication:**
Ensure secure login mechanisms in the web application.
 - **Data Protection:**
Maintain proper handling and storage of user and system data to prevent unauthorized access.

8.2 System Evaluation

System evaluation is carried out to measure the performance and effectiveness of the Smart Automatic Irrigation System. It ensures that the system meets its objectives and performs efficiently under different conditions.

1.Evaluation Parameters

- **Accuracy:**
The system is evaluated based on how accurately it predicts irrigation needs using sensor data and machine learning.
- **Efficiency:**
Measures how effectively the system reduces water wastage and optimizes irrigation.
- **Response Time:**
Evaluates how quickly the system responds to changes in soil moisture conditions.
- **Reliability:**
Checks whether the system operates continuously without failure.
- **Usability:**
Assesses how easily users can interact with the dashboard and understand system outputs.

❖ Performance Evaluation

The system successfully monitors real-time soil moisture and environmental conditions. The machine learning model provides reliable predictions for irrigation. The automatic control mechanism ensures timely activation and deactivation of the water pump. The web dashboard provides clear and accessible system information.

❖ **Limitations Identified**

The system depends on sensor accuracy, which may vary due to environmental factors. Machine learning predictions depend on the quality of training data. Continuous power supply is required for uninterrupted operation.

❖ **Future Improvements**

Integration with IoT platforms for remote monitoring through mobile applications. Use of advanced sensors for more accurate data collection. Enhancement of machine learning models using larger datasets. Addition of weather-based irrigation control for better efficiency.

**COST AND BENEFIT
ANALYSIS**

9. COST AND BENEFIT ANALYSIS

9.1 Costs

A. Initial Costs:

1. Development Costs:

- **Software Development:**

Costs involved in designing and developing the Django web application, implementing backend logic, and integrating the machine learning model for irrigation prediction.

- **Model Development:**

Effort required for collecting dataset, training the machine learning model, and generating the prediction system used in irrigation decisions.

- **Tools and Setup:**

Includes development environments, coding tools, and initial system configuration. Most tools used are open-source, which reduces cost.

2. Hardware Costs:

- **Controller and Sensors:**

Cost of NodeMCU (ESP8266), soil moisture sensor, DHT11 sensor, and ultrasonic sensor used for data collection.

- **Control Components:**

Includes relay module and buzzer used for controlling and alerting system operations.

- **Water Pump:**

Cost of motor used for supplying water to plants.

- **Power Supply and Accessories:**

Includes AC adapter, connecting wires, and other circuit components required for setup.

3. Installation Costs

- **System Setup:**

Expenses related to assembling hardware components and establishing connections.

- **Configuration:**

Initial setup of software system, database, and integration with hardware components.

4. Training Costs

- **User Training:**

Minimal cost required to train users on how to operate and monitor the system using the web dashboard.

B. Recurring Costs:

1. Operational Costs:

- **Maintenance and Updates:**

Regular updates to the Django application, bug fixing, and system improvements.

- **Electricity Consumption:**

Power required to operate sensors, Node MCU, and water pump.

2. Support Costs:

- **System Monitoring:**

Effort required to check system performance and handle any issues.

- **Hardware Maintenance:**

Periodic checking and replacement of sensors or components if required.

9.2 Benefits:

1. Water Conservation:

The system uses water efficiently by supplying it only when required, reducing wastage and supporting sustainable usage.

2. Reduced Manual Effort:

Automation eliminates the need for manual watering, saving time and labor for users.

3. Improved Plant Growth:

Maintaining proper soil moisture ensures better plant health and increases productivity.

4. Intelligent Decision-Making:

The use of machine learning improves accuracy in irrigation decisions compared to traditional methods.

5. Cost Efficiency:

Although there is an initial setup cost, the system reduces long-term expenses by saving water and minimizing labor.

6. Real-Time Monitoring:

The web dashboard allows users to monitor system data and irrigation status anytime, improving control and management.

7. Scalability and Future Expansion:

The system can be extended with additional features such as IoT integration, mobile applications, and weather-based irrigation.

OVERALL ANALYSIS:

The Smart Automatic Irrigation System provides significant benefits compared to its cost. The initial investment is moderate, but the long-term advantages such as water saving, reduced labor, and improved efficiency make the system highly beneficial and practical.

**DETAILED LIFE CYCLE
OF THE PROJECT**

10.DETAILED LIFE CYCLE OF THE PROJECT

10.1 Life Cycle Phases

Conceptualization:

- **Idea Generation:**

- **Problem Identification:** Identify the issue of manual irrigation leading to water wastage and inefficient plant care.
- **Solution Idea:** Develop an automatic irrigation system using sensors and machine learning to optimize water usage.
- **Technology Selection:** Use IoT components (NodeMCU, sensors) and software technologies (Django, ML model) for system development.

- **Feasibility Study:**

- **Technical Feasibility:** Evaluate the ability to integrate sensors, NodeMCU, machine learning model, and web application.
- **Economic Feasibility:** Analyze cost of hardware components and development tools; most software tools are open-source, reducing cost.
- **Operational Feasibility:** Ensure that the system is easy to use and can operate automatically with minimal human intervention.

- **Project Scope Definition:**

- **Objectives:** Automate irrigation, reduce water wastage, and improve plant health using intelligent decision-making.
- **Features:** Soil moisture detection, automatic pump control, ML-based prediction, and web-based monitoring dashboard.

1.Planning:

- **Project Plan Development:**

- **Timeline:** Define phases such as requirement analysis, design, implementation, testing, and deployment.
- **Resource Allocation:** Assign hardware components, development tools, and team responsibilities.

2.Risk Management:

- **Risk Identification:** Possible risks include sensor failure, incorrect predictions, power supply issues, and system errors.
- **Mitigation Strategies:** Regular testing, proper hardware maintenance, backup power solutions, and model improvement.

3. System Design

- **Architecture Design:**

- **Hardware Design:** Design connections between sensors, NodeMCU, relay module, and water pump.
- **Software Design:** Design Django-based backend, database structure, and user interface.

- **Data Design:**

- **Input Data:** Soil moisture, temperature, humidity, and water level.
- **Output Data:** Irrigation decision (Pump ON/OFF) and dashboard display.

4. Implementation

- **Hardware Implementation:** Connect sensors (moisture, DHT11, ultrasonic) to NodeMCU. Integrate relay module with water pump.
- **Software Implementation:** Develop backend using Django. Implement machine learning model for prediction. Design frontend dashboard for monitoring.

5. Testing

- **Unit Testing:** Test individual components such as sensors and ML model.
- **System Testing:** Verify complete system functionality including data flow and pump control.
- **Performance Testing:** Check response time and accuracy of predictions.

6. Deployment

- **System Installation:** Install hardware components in real environment.
- **System Configuration:** Configure software and connect it with hardware.

6. User Access:

Provide access to dashboard for monitoring system.

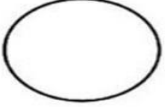
7. Maintenance:

- **System Monitoring:** Continuously monitor system performance and sensor readings.
- **Updates and Improvements:** Update software and retrain ML model when required.
- **Hardware Maintenance:** Regular checking of sensors, relay, and pump for proper working.

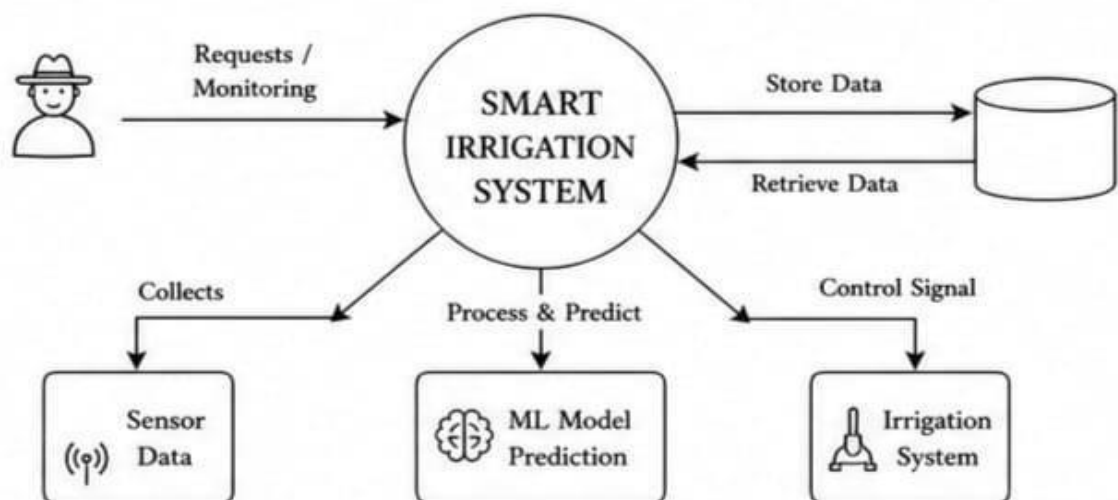
10.2 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the "flow" of data through an information system (as shown on the DFD flow chart Figure 5), Modeling its process aspects. Often it is a preliminary step used to create an overview of the system that can later be elaborated.

 **Symbols used in dataflow diagram are:**

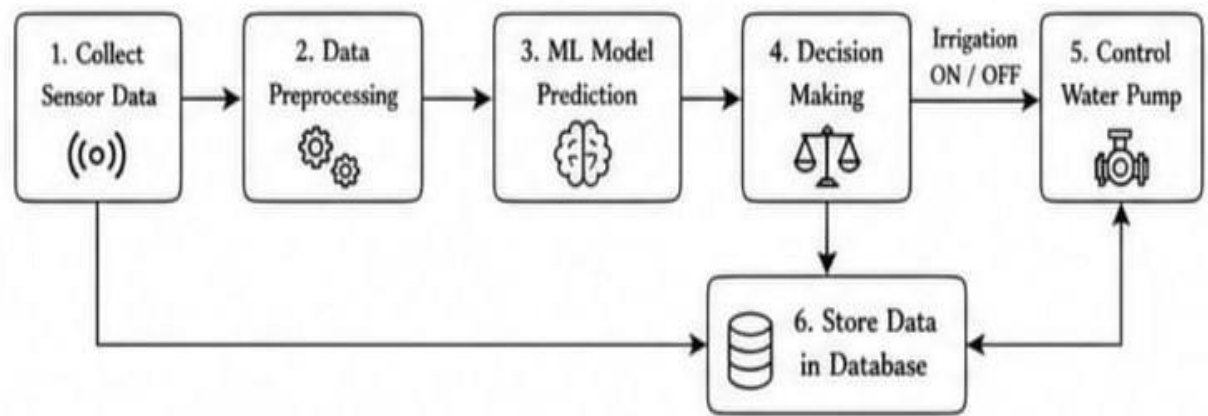
Symbol	Name	Function
	Data flow	Used to Connect Processes to each other, to sources or Sinks; the arrow head indicates direction of data flow.
	Process	Performs Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

 **0 Level DFD**

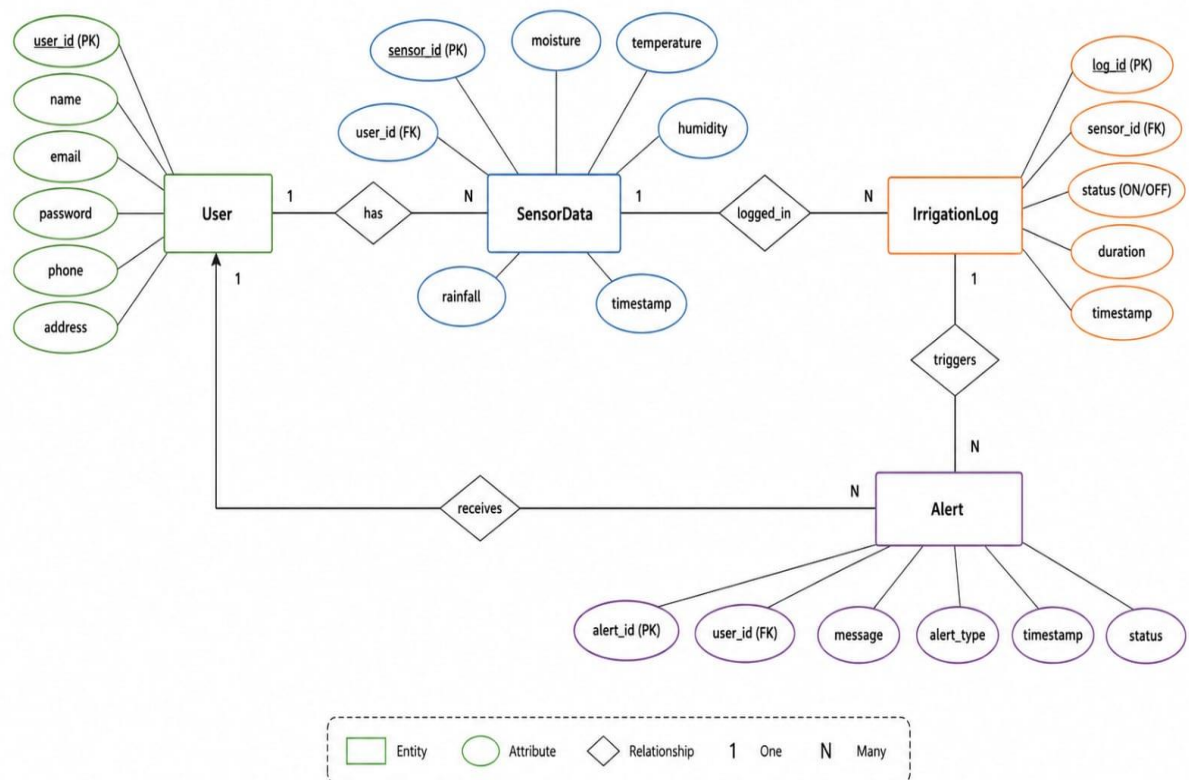




DFD Level 1

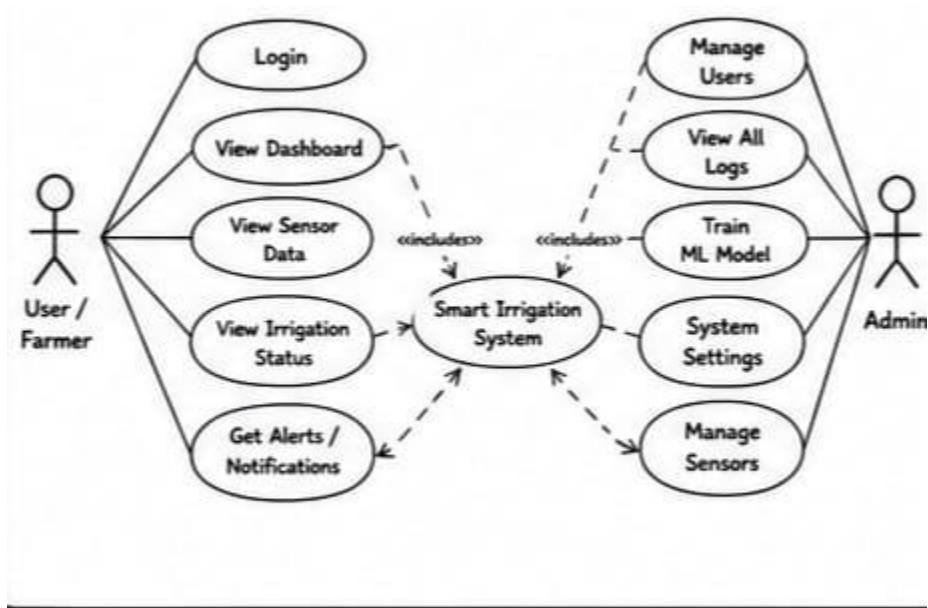


ER Diagram

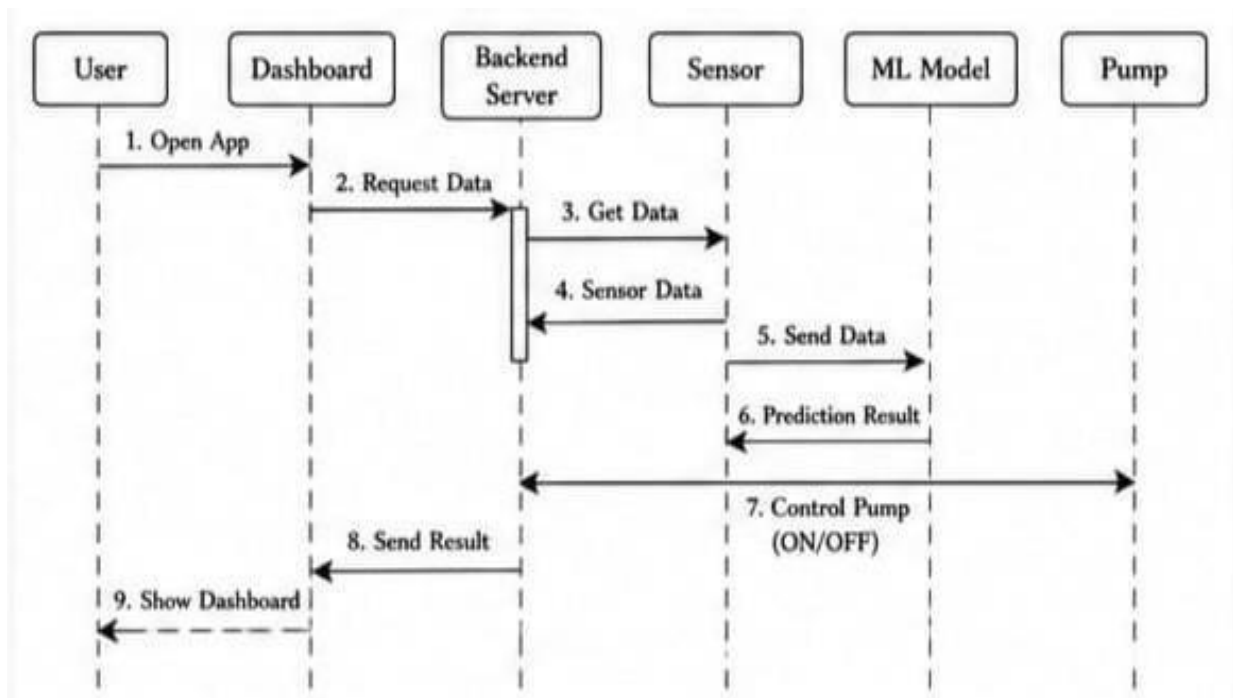


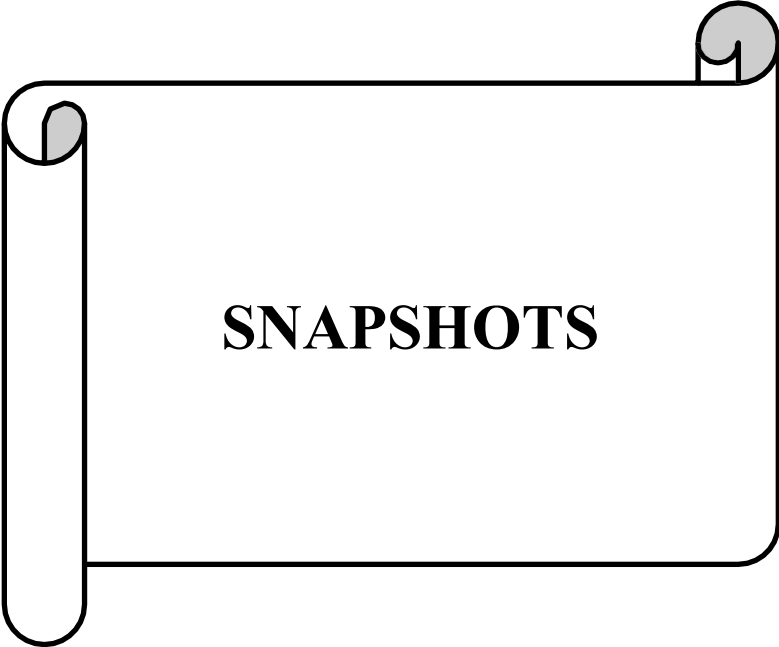


Use case Diagram



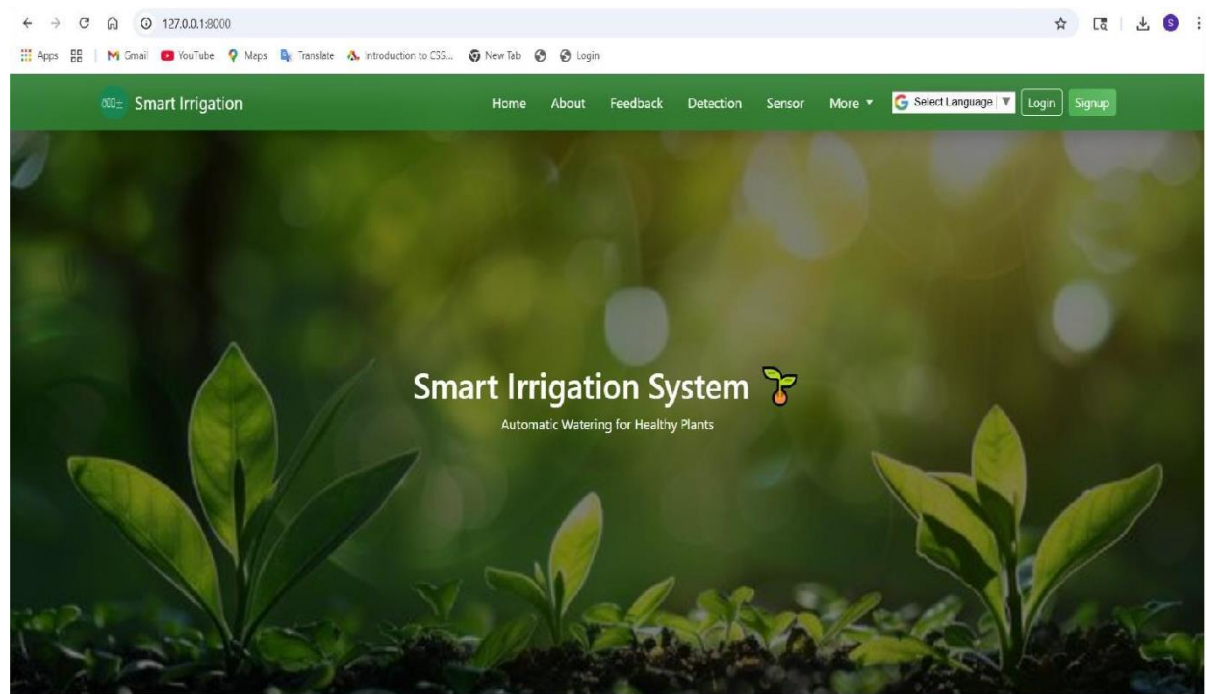
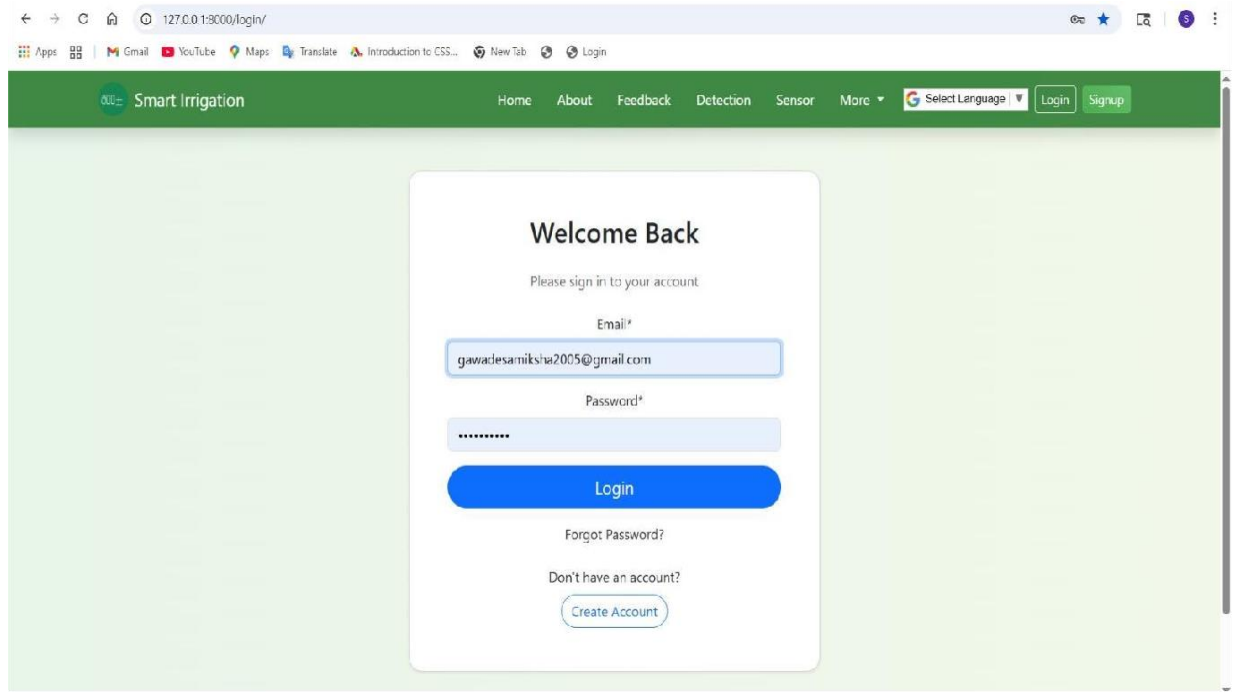
Sequence Diagram

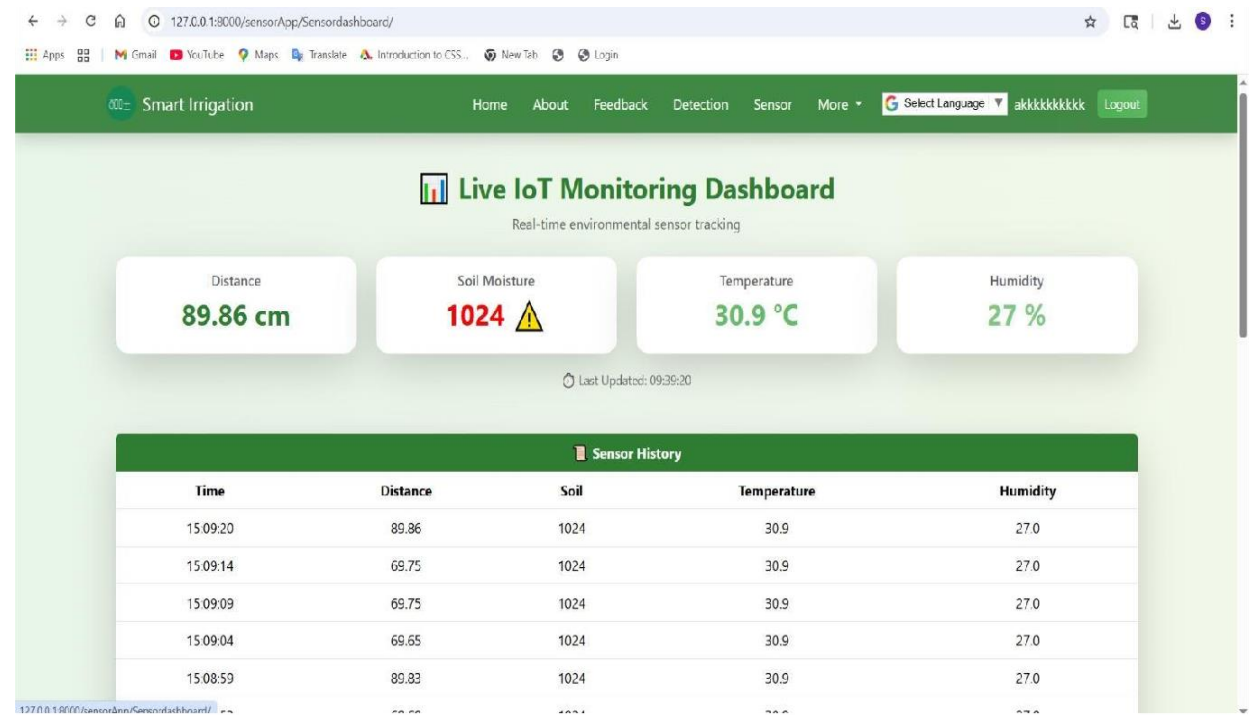
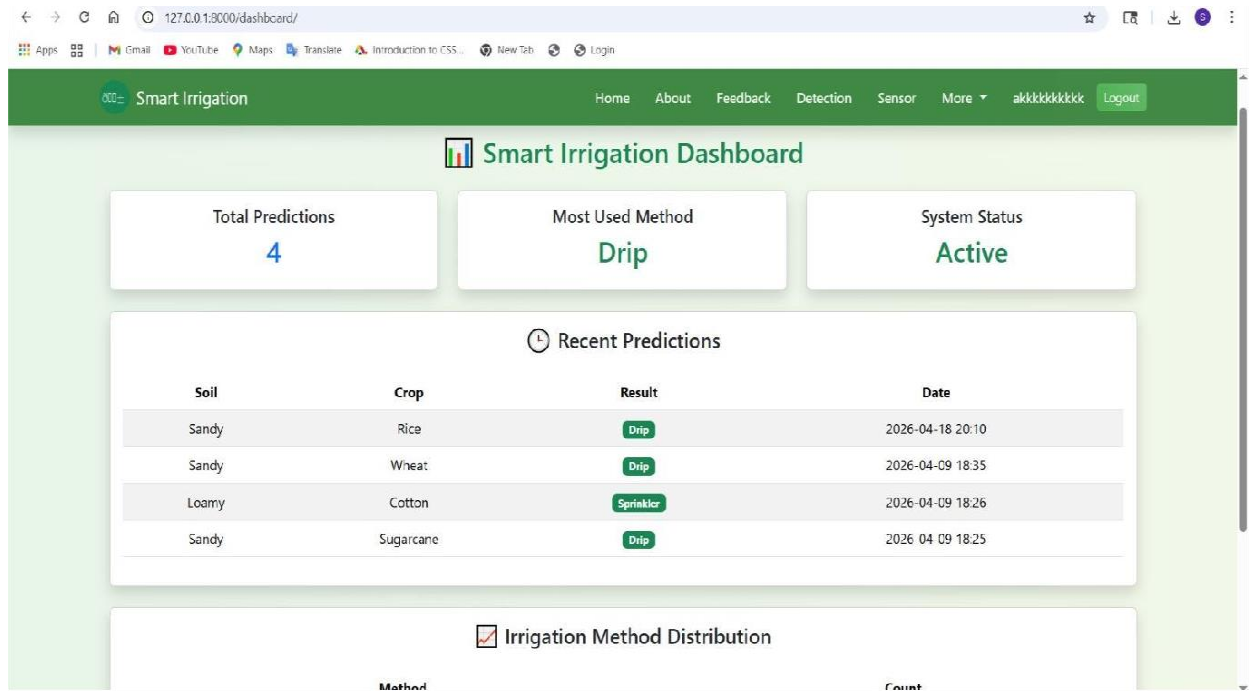




SNAPSHOTS

11.SNAPSHOTS





127.0.0.1:3000/modelapp/predict/

Smart Irrigation

Home About Feedback Detection Sensor More Select Language akkkkkkkkk Logout

Irrigation Prediction System

Soil Type

Select Soil Type

Crop Type

Select Crop

Temperature (°C)

Enter temperature (0-50°C)

Humidity (%)

Enter humidity (0-100%)

Rainfall (mm)

Enter rainfall (0-500 mm)

Water Availability

Select Water Level

Predict Irrigation

127.0.0.1:3000/feedback/

Smart Irrigation

Home About Feedback Detection Sensor More Select Language akkkkkkkkk Logout

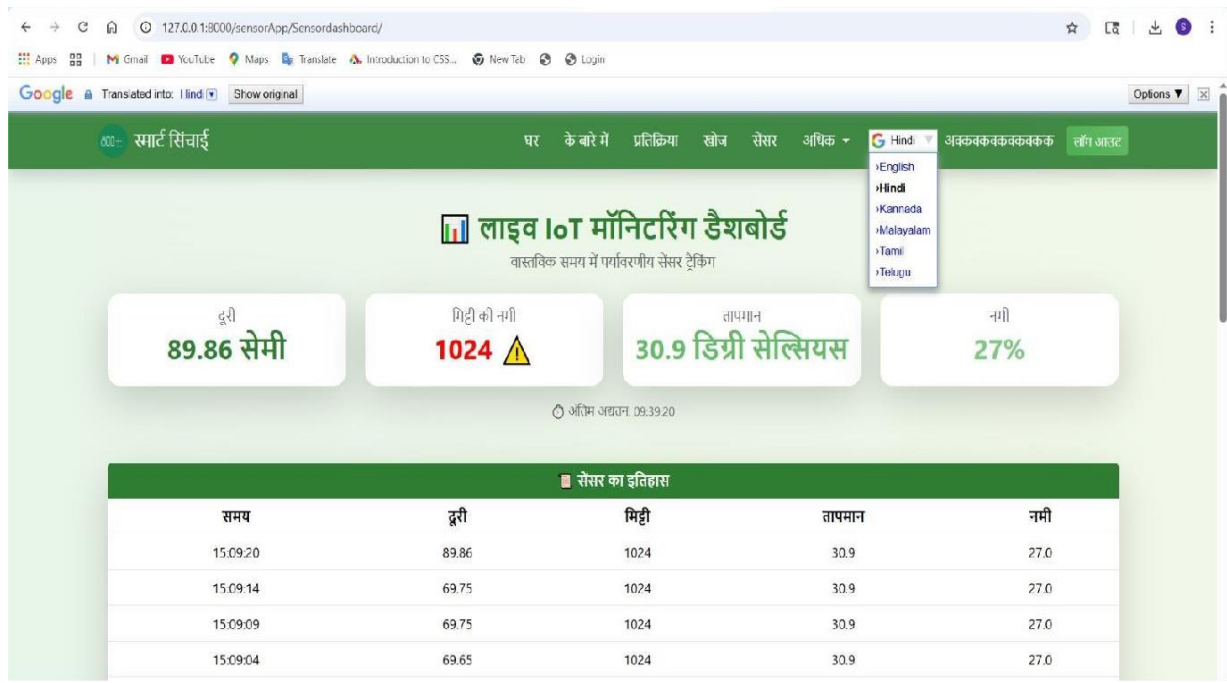
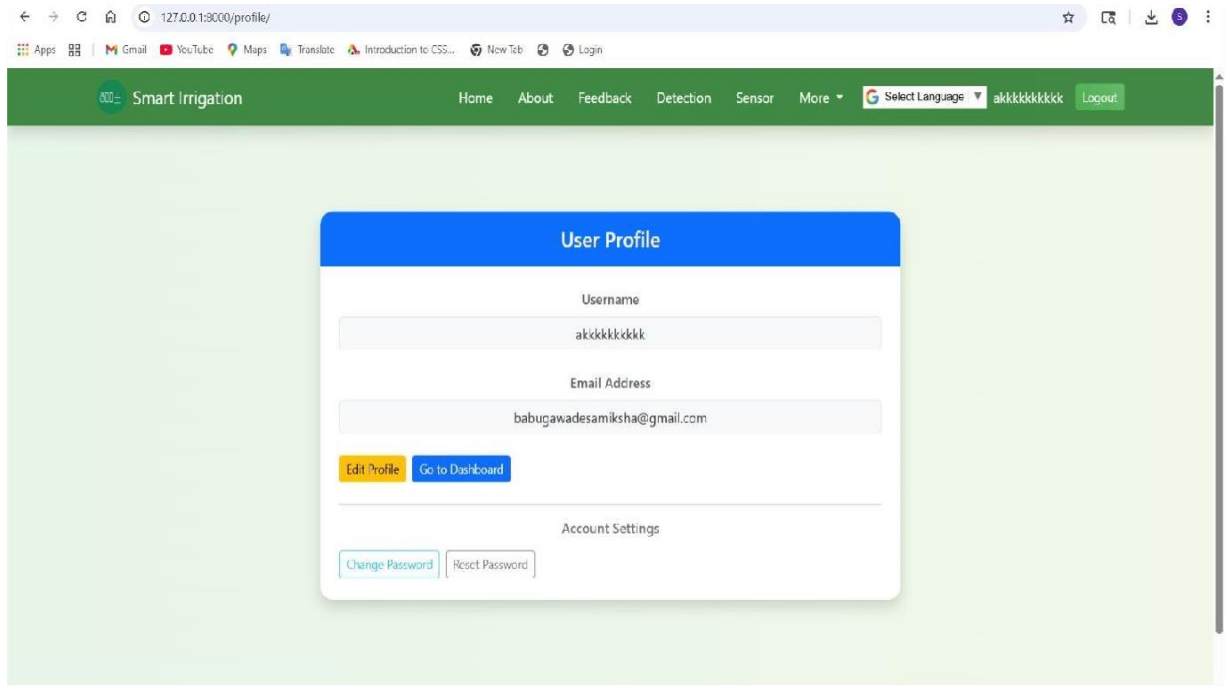
Submit Feedback

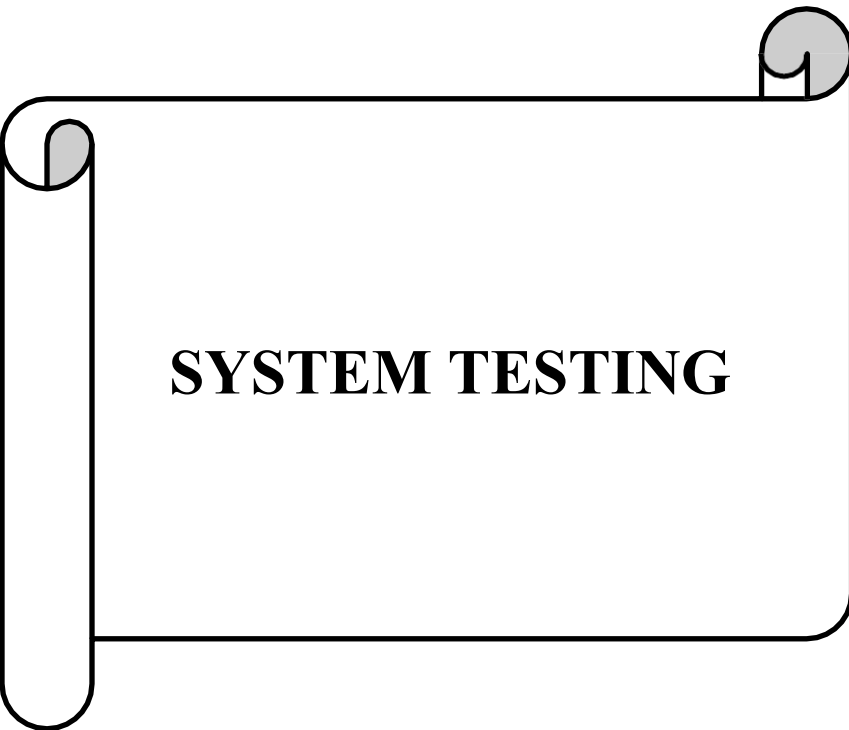
Your Feedback*

Write your feedback here...

Submit Feedback

© 2026 Smart Irrigation System





SYSTEM TESTING

12.SYSTEM TESTING

System testing plays a crucial role in ensuring the proper functioning, reliability, and efficiency of the Smart Automatic Irrigation System. Since the system integrates IoT hardware, machine learning, and a web-based application, testing is necessary to verify that all components work together correctly and deliver accurate irrigation control.

- **Understanding System Testing**

System testing involves evaluating the complete system as a whole. In the Smart Automatic Irrigation System, it includes testing the hardware components (sensors, Node MCU, relay, pump), software application (Django-based system), and machine learning model. The objective is to ensure smooth interaction between all modules and proper system performance under different conditions.

- **Key Objectives of System Testing**

- **Functionality Testing**

This testing ensures that all features of the system work correctly.

- Verifies soil moisture detection and data collection
- Checks automatic pump ON/OFF operation based on conditions
- Ensures proper working of sensors (temperature, humidity, water level)
- Validates display of data on the web dashboard

- **Performance Testing**

This testing evaluates the system's performance under different conditions.

- Checks system response time to sensor inputs
- Ensures real-time data processing without delay
- Verifies system stability during continuous operation
- Tests performance when handling multiple sensor readings

- **Usability Testing**

This testing focuses on user interaction with the system.

- Ensures that the dashboard is easy to use and understand
- Verifies clear display of sensor data and system status
- Checks user accessibility and navigation within the application

- **Security Testing**

This testing ensures protection of system data.

- Verifies secure user authentication and login
- Ensures protection of stored data in the database
- Prevents unauthorized access to system controls

- **Integration of Machine Learning and IoT**

The integration of machine learning and IoT enhances the intelligence of the system. System testing ensures proper coordination between hardware and software components.

- **Accuracy Validation**

- Verifies accuracy of machine learning predictions
- Ensures correct irrigation decisions based on sensor data
- Tests system behavior under different environmental conditions

- **Compatibility Testing**

- Ensures smooth communication between NodeMCU and Django system
- Verifies compatibility between hardware components and software modules
- Tests system functionality across different devices and browsers

- **Testing Process Overview**

- **Test Planning**

- Define testing objectives, scope, and strategies
- Prepare test cases based on system requirements
- Identify tools and methods for testing

- **Test Execution**

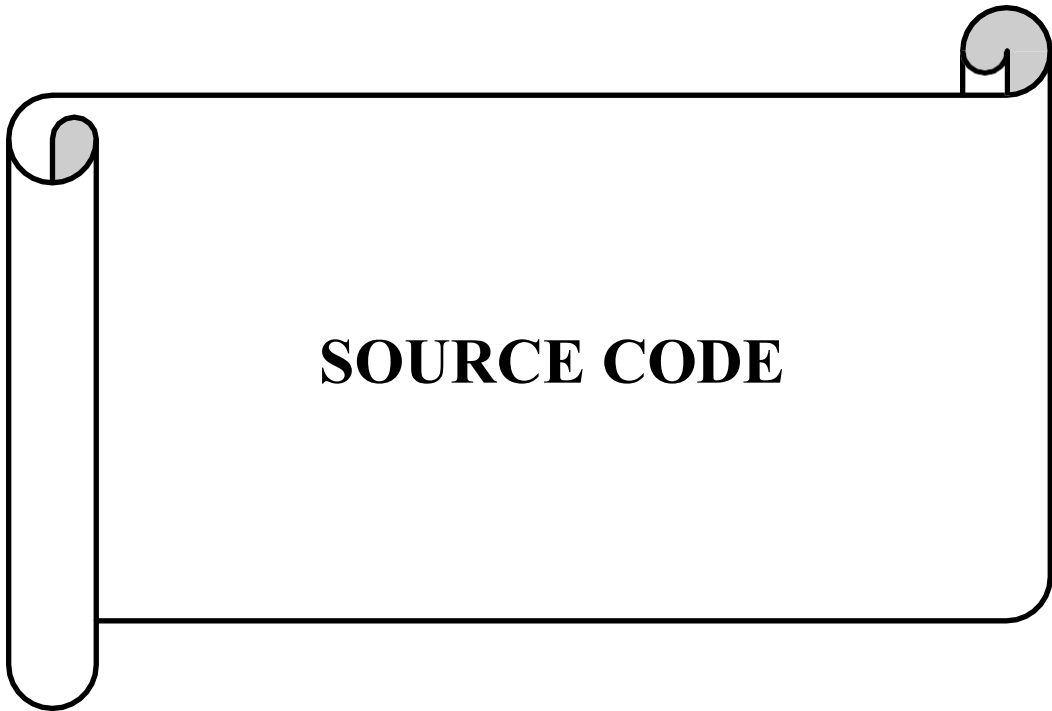
- Perform hardware testing (sensor readings, pump operation)
- Conduct software testing (dashboard, database operations)
- Test machine learning predictions with sample data

- **Test Evaluation**

- Analyze results to identify errors or mismatches
- Check performance issues and system limitations
- Improve system based on feedback

- **Regression Testing**

- Re-test the system after updates or modifications
- Ensure new changes do not affect existing functionalities
- Maintain system stability after improvement



SOURCE CODE

13.SOURCE CODE

➤ **project.ino**

```
#include <ESP8266WiFi.h>
#include
<ESP8266HTTPClient.h>
#include <DHT.h>

// ===== Pin Definitions
===== #define TRIG D1
#define ECHO
D2 #define SOIL
A0 #define
DHTPIN D5
#define BUZZER
D6 #define
RELAY D7
#define DHTTYPE DHT11

// ===== WiFi Credentials
===== const char* ssid =
"BSNL3G";
const char* password = "Asdf1234..";

// ===== Django Server =====
const char* serverName = "http://10.133.253.204:8000/sensorApp/api/sensor/";

// ===== Objects =====
DHT dht(DHTPIN, DHTTYPE);

void setup() {
Serial.begin(11520
0);

// Pin Modes
pinMode(TRIG,
OUTPUT);
pinMode(ECHO,
INPUT);

pinMode(BUZZER,
OUTPUT);
pinMode(RELAY,
OUTPUT);
```

```

// Initial States
digitalWrite(BUZZER,
LOW);
digitalWrite(RELAY,
HIGH);

// WiFi Connect
WiFi.begin(ssid,
password);

while (WiFi.status() != WL_CONNECTED) {
delay(1000);
Serial.println("Connecting to WiFi...");
}

Serial.println("✅
Connected!");
Serial.print("NodeMCU IP: ");
Serial.println(WiFi.localIP());

dht.begin();
}

void loop() {

// ===== Ultrasonic Sensor =====
digitalWrite(TRIG, LOW); delayMicroseconds(2);

digitalWrite(TRIG,
HIGH);
delayMicroseconds(10);
digitalWrite(TRIG,
LOW);

long duration = pulseIn(ECHO, HIGH, 30000);
float distance = (duration == 0) ? -1 : duration * 0.034 / 2;

// ===== Soil Moisture
===== int soil =
analogRead(SOIL);

// ===== DHT11 =====
float temp = dht.readTemperature();
float hum = dht.readHumidity();

// Fix NaN values
if (isnan(temp) || isnan(hum)) {
Serial.println("❌ DHT Error!");
temp = 0;

```

```

        hum = 0;
    }

    // ===== Buzzer Logic =====
    // Condition: distance > 5 AND soil dry
    if (distance > 5 && distance < 1 && soil >= 1000) {
digitalWrite(BUZZER, HIGH);
    } else {
digitalWrite(BUZZER, LOW);
    }

    // ===== Relay Logic =====
    // Condition: distance between 1 and 5 AND soil dry
    // ===== Relay Logic (FIXED) =====
    if (distance >= 1 && distance <= 5 && soil >= 1000) {
delay(2000);
digitalWrite(RELAY, LOW); // ON (Active LOW)
    } else {
digitalWrite(RELAY, HIGH); // OFF
    }

    // ===== Serial Debug =====
    Serial.println("----- Sensor Data
    -----");
);

    Serial.print("Distance: ");
    Serial.print(distance);
    Serial.println(" cm");

    Serial.print("Soil: ");
    Serial.println(soil);

    Serial.print("Tem
p: ");
    Serial.print(temp);
    Serial.println("
°C");

    Serial.print("Humidity:
"); Serial.print(hum);
    Serial.println(" %");

    Serial.println("-----");
    // ===== Send Data to Django =====
    if (WiFi.status() == WL_CONNECTED) {

        HTTPClient

```

```
http;
WiFiClient
client;

http.begin(client, serverName);
http.addHeader("Content-Type", "application/json");

String json = "{";
json += "\"distance\":" + String(distance) + ",";
json += "\"soil\":" + String(soil) + ",";
json += "\"temperature\":" + String(temp) + ",";
json += "\"humidity\":" + String(hum);
json += "}";

Serial.println("📡 Sending JSON:");
Serial.println(json);

int httpResponseCode = http.POST(json);

Serial.print("HTTP Response: ");
Serial.println(httpResponseCode);

String response = http.getString();
Serial.println("Server Response:");
Serial.println(response);

http.end();
}

delay(5000);
}
```

➤ **Base.html**

```
{% load static %}
<!DOCTYPE html>
<html lang="en" data-bs-theme="light">

<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Smart Irrigation</title>

<!-- Bootstrap CSS -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

<!-- Bootstrap Icons -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.css" rel="stylesheet">

<!-- 🌿 GREEN INTERACTIVE STYLE -->
<style>
/* BODY */ body
{
background: linear-gradient(to right, #e8f5e9, #f1f8e9);
}

/* NAVBAR */
.navbar {
background: rgba(46, 125, 50, 0.9) !important;
backdrop-filter: blur(12px);
border-bottom: 1px solid rgba(255,255,255,0.2); transition:
0.3s;
}

.navbar:hover {
background: rgba(27, 94, 32, 0.95) !important;
}

/* BRAND */
.navbar-brand span {
color: white;
font-size: 18px; letter-
spacing: 1px;
}

/* NAV LINKS */
.nav-link {
```

```
    color: white !important;
font-weight: 500; position:
relative; transition: 0.3s;
}
```

```
/* HOVER LINE */
.nav-link::after {
content: ""; position:
absolute; width: 0%;
height: 2px; bottom: 0;
left: 0;
background: #81c784;
transition: 0.3s;
}
```

```
.nav-link:hover::after {
width: 100%;
}
```

```
.nav-link:hover {
color: #a5d6a7 !important;
transform: translateY(-2px);
}
```

```
/* ACTIVE */
.nav-link.active::after {
width: 100%;
}
```

```
/* GREEN BUTTON */
.btn-custom {
background: linear-gradient(45deg, #4caf50, #66bb6a);
border: none;
color: white;
font-weight: 600;
transition: 0.3s;
}
```

```
.btn-custom:hover {
transform: scale(1.05);
box-shadow: 0 0 12px #4caf50;
}
```

```
/* PROFILE */
.profile-img {
```

```

    width: 32px; height:
32px; border-radius:
50%; object-fit: cover;
    }

    /* BREADCRUMB */
    .breadcrumb a {
color: #2e7d32;
    text-decoration: none;
    }

    /* FOOTER */
    footer {
background: #c8e6c9 !important;
    }

    .footer-text { color:
#1b5e20;
    font-weight: 600;
letter-spacing: 1px;
    }
</style>
</head>

<body>

<!-- NAVBAR START -->
<nav class="navbar navbar-expand-lg shadow sticky-top">
<div class="container">

    <!-- Brand -->
    <a class="navbar-brand fw-bold d-flex align-items-center gap-2" href="/">
    
    <span>Smart Irrigation</span>
    </a>

    <!-- Toggle -->
    <button class="navbar-toggler bg-light" type="button" data-bs-toggle="collapse" data-bs-
target="#mainNavbar">
    <span class="navbar-toggler-icon"></span>
    </button>

    <div class="collapse navbar-collapse" id="mainNavbar">

```

```

<!-- Menu -->
<ul class="navbar-nav ms-auto align-items-lg-center gap-lg-3">

  <li class="nav-item">
    <a class="nav-link {% if request.path == '/' %}active{% endif %}" href="/">
      <i class="bi bi-house-door"></i> Home
    </a>
  </li>

  <li class="nav-item">
    <a class="nav-link {% if request.resolver_match.url_name == 'about' %}active{% endif %}"
href="{% url 'about' %}">
      <i class="bi bi-info-circle"></i> About
    </a>
  </li>

  <li class="nav-item">
    <a class="nav-link" href="{% url 'feedback' %}">
      <i class="bi bi-chat-dots"></i> Feedback
    </a>
  </li>

  <li class="nav-item">
    <a class="nav-link" href="{% url 'modelapp:predict' %}">
      <i class="bi bi-cpu"></i> Detection
    </a>
  </li>

  <li class="nav-item">
    <a class="nav-link" href="{% url 'SensorApp:sensorDashboard' %}">
      <i class="bi bi-moisture"></i> Sensor
    </a>
  </li>

  <!-- Dropdown -->
  <li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" href="#" data-bs-toggle="dropdown"> More
    </a>

    <ul class="dropdown-menu dropdown-menu-end shadow">

      <li>
        <a class="dropdown-item" href="{% url 'dashboard' %}">
          <i class="bi bi-speedometer2"></i> Dashboard
        </a>
      </li>

```

```

</li>
<a class="dropdown-item" href="{% url 'user_list' %}">
<i class="bi bi-envelope"></i> Messages
</a>
</li>

</li>
<a class="dropdown-item" href="{% url 'modelapp:history' %}">
<i class="bi bi-clock-history"></i> History
</a>
</li>

<li><hr class="dropdown-divider"></li>

</li>
<a class="dropdown-item d-flex align-items-center gap-2" href="{% url 'profile' %}">
{% if request.user.profile_pic %}

{% else %}
<i class="bi bi-person-circle"></i>
{% endif %} Profile
</a>
</li>

</ul>
</li>
</ul>

<!-- Right Side -->
<div class="d-flex align-items-center gap-2 ms-lg-3">

{% if user.is_authenticated %}

<div class="d-flex align-items-center gap-2 text-white">
{% if request.user.profile_pic %}

{% else %}
<i class="bi bi-person-circle fs-5"></i>
{% endif %}
<small>{{ user.username }}</small>
</div>

<form method="post" action="{% url 'logout' %}">
{% csrf_token %}
<button class="btn btn-custom btn-sm"> Logout

```

```

</button>
</form>

{% else %}

<a href="{% url 'login' %}" class="btn btn-outline-light btn-sm"> Login
</a>

<a href="{% url 'signup' %}" class="btn btn-custom btn-sm"> Signup
</a>

{% endif %}

<!-- Theme Toggle -->
<button class="btn btn-outline-light btn-sm rounded-circle" id="themeToggle">
<i class="bi bi-moon-stars"></i>
</button>

</div>
</div>
</div>
</nav>

<!-- BREADCRUMB -->
<div class="container mt-3">
<nav>
<ol class="breadcrumb">
<li class="breadcrumb-item"><a href="/">Home</a></li>
<li class="breadcrumb-item active">
{{ request.resolver_match.url_name|title }}
</li>
</ol>
</nav>
</div>

<!-- MAIN CONTENT -->
<div class="container text-center mt-4">

<!-- 🌱 PLANT IMAGE ADDED -->


<div class="row justify-content-center">
<div class="col-lg-10">

```

```

    {% block content %}
    {% endblock %}
  </div>
</div>

<!-- FOOTER -->
<footer class="border-top py-4 mt-auto">
  <div class="container text-center">
    <span class="footer-text">
      © 2026 Smart Irrigation System 🌱
    </span>
  </div>
  >
</fo
ter>

<!-- JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

<script>
  const toggleBtn =
document.getElementById("themeToggle"); const icon =
toggleBtn.querySelector("i");

  function setTheme(theme) {
document.documentElement.setAttribute("data-bs-theme", theme);
localStorage.setItem("theme", theme);

    if (theme === "dark") {
      icon.classList.replace("bi-moon-stars", "bi-sun");
    } else {
      icon.classList.replace("bi-sun", "bi-moon-stars");
    }
  }

  toggleBtn.addEventListener("click", () => {
    const current = document.documentElement.getAttribute("data-bs-theme");
    setTheme(current === "dark" ? "light" : "dark");
  });

  window.onload = () => {
    setTheme(localStorage.getItem("theme") || "light");
  };
</script>

</body>
</html>

```

➤ **Models.py**

```
from django.db

import models class

SensorData(models.

Model):
    distance = models.FloatField()
    soil = models.IntegerField()
    temperature = models.FloatField(null=True,
blank=True) humidity =
models.FloatField(null=True, blank=True)
    created_at =
models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.created_at} - {self.distance} cm'

class
Alert(models.Model):
    distance =
models.FloatField()
    soil =
models.IntegerField()
    message =
models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'Alert at {self.created_at}'
```

About.html

```
{% extends "base.html" %}
{% load static %}

{% block content %}

<!-- About Section -->
<section class="py-5">
<div class="container">
<div class="row align-items-center">

<!-- Left Image -->
<div class="col-lg-6 col-md-12 mb-4 mb-lg-0">
width="85%"
```

```
  
<div class="col-lg-6 col-md-12">  
  <p class="text-uppercase fw-semibold mb-2" style="color: green;">  
About The System  
  </p>
```

```
  <h2 class="fw-bold mb-4" style="color:  
darkgreen;"> AI-Powered Prediction Platform  
  </h2>
```

```
  <p style="color: #2e7d32;">  
    This platform leverages intelligence to assist professionals  
in predicting the risk of disorders.  
    It supports diagnosis, decision-making, and tracking through  
smart analytics.  
  </p>
```

```
  <ul class="list-unstyled mt-4 mb-4" style="color: #2e7d32;">  
    <li class="mb-2">🔍 Predicts and more.</li>  
    <li class="mb-2">🔍 Supports and more.</li>  
    <li class="mb-2">🔍 Secure, data-driven insights with intuitive dashboard</li>  
  </ul>
```

```
  <a href="{% url 'feedback' %}" class="btn btn-success px-4 py-2">  
Contact Us  
  </a>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</section>
```

```
{% endblock %}
```


ANNEXURE

14. ANNEXURE

The annexure section provides supporting technical details of the Smart Automatic Irrigation System. It includes the data dictionary, abbreviations, figures, and other reference information required for better understanding of the system.

14.1 DATA DICTIONARY

A Data Dictionary is a structured repository that defines all data elements used in the system. It provides details about database tables, fields, data types, and relationships, ensuring consistency and clarity in system design.

1. User Table

Field Name	Data Type	Description
user_id (PK)	Integer	Unique identifier for each user
name	Varchar	Name of the user
email	Varchar	Email used for login
password	Varchar	Encrypted user password
phone	Varchar	Contact number
address	Text	User location details

2. SensorData Table

Field Name	Data Type	Description
sensor_id (PK)	Integer	Unique ID for sensor data
user_id (FK)	Integer	Reference to User table
moisture	Float	Soil moisture level
temperature	Float	Temperature reading
humidity	Float	Humidity value
rainfall	Float	Rain sensor reading
timestamp	DateTime	Date and time of data

3. IrrigationLog Table

Field Name	Data Type	Description
log_id (PK)	Integer	Unique log identifier
sensor_id (FK)	Integer	Reference to sensor data
status	Boolean	Pump ON/OFF status
duration	Float	Irrigation duration
timestamp	DateTime	Time of irrigation

4. Alert Table

Field Name	Data Type	Description
alert_id (PK)	Integer	Unique alert ID
user_id (FK)	Integer	Reference to user
message	Text	Alert message
alert_type	Varchar	Type of alert
timestamp	DateTime	Alert time
status	Boolean	Read/Unread status

14.2 IMPORTANCE OF DATA DICTIONARY

- Helps in managing complex system data efficiently
- Provides a clear understanding of database structure
- Ensures consistency in naming and data usage
- Supports system development and maintenance
- Helps in detecting errors and missing data
- Improves communication among developers and Users

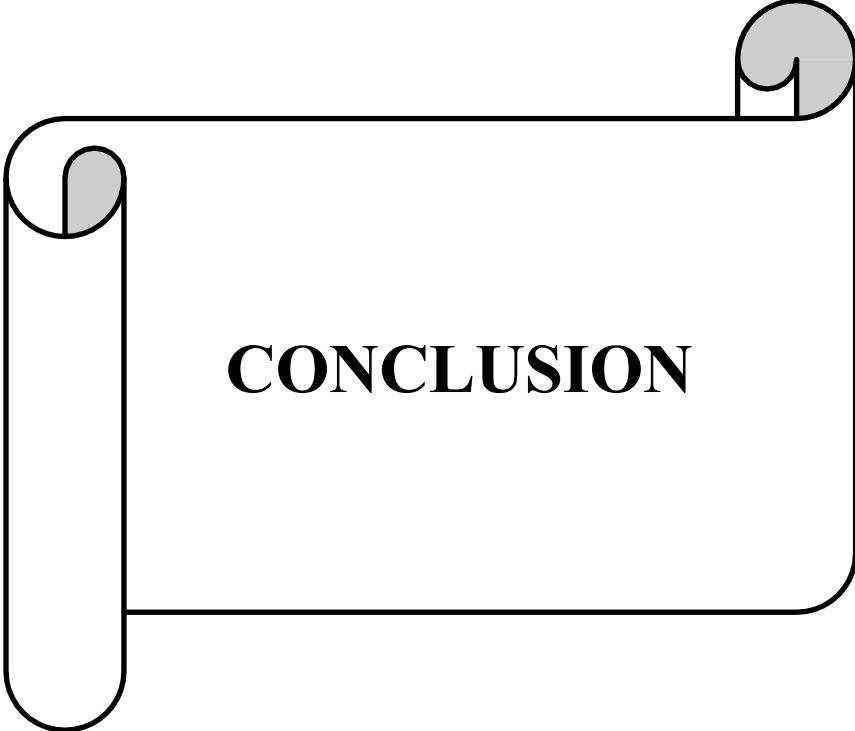
14.3 ABBREVIATIONS

- HTML – HyperText Markup Language
- CSS – Cascading Style Sheets
- DFD – Data Flow Diagram
- ER – Entity Relationship
- ML – Machine Learning
- IoT – Internet of Things
- DBMS – Database Management System
- API – Application Programming Inter face

14.4 FIGURES

- System Architecture Diagram
- Data Flow Diagram (Level 0 and Level 1)
- ER Diagram (Database Design)
- Use Case Diagram
- Sequence Diagram
- Flowchart
- Gantt Chart
- Spiral Model

These figures represent the overall working, structure, and data flow of the Smart Automatic Irrigation System.



CONCLUSION

15. CONCLUSION

The Smart Automatic Irrigation System is an efficient and intelligent solution developed using IoT, Machine Learning, and Web Technologies. The system continuously monitors environmental parameters such as soil moisture, temperature, humidity, and water level to make accurate irrigation decisions.

The integration of machine learning improves the system's ability to predict irrigation requirements, reducing water wastage and human effort. The Django-based web interface allows users to monitor system performance and irrigation status in real time.

The system has been successfully implemented and tested, demonstrating reliable performance under different conditions. It provides a practical and scalable solution for modern agriculture, contributing to water conservation and improved crop productivity.

In addition to automation, the system enhances precision in irrigation by ensuring that water is supplied only when required. This prevents both over-irrigation and under-irrigation, which are common problems in traditional methods. As a result, the system helps in maintaining optimal soil conditions for plant growth.

The integration of IoT devices enables real-time data collection and communication between hardware and software components. This improves decision-making speed and ensures timely irrigation actions. The use of a centralized web dashboard further simplifies monitoring and control, making the system user-friendly and accessible.

The system also demonstrates the effective use of modern technologies in solving real-world agricultural problems. By combining sensor-based monitoring with intelligent prediction, it reduces dependency on manual labor and increases operational efficiency.

Overall, the Smart Automatic Irrigation System proves to be a reliable, cost-effective, and sustainable solution. It has the potential to be implemented on a larger scale in agricultural fields, gardens, and smart farming environments, contributing to efficient water management and improved agricultural practices.

**FUTURE
ENHANCEMENTS**

16. FUTURE ENHANCEMENTS

- **Advanced Sensors:**
Use high-accuracy sensors for better data collection
- **Weather Forecast Integration:**
Integrate weather APIs to improve irrigation decisions
- **Mobile Application:**
Develop a mobile app for easy monitoring and control
- **Cloud Integration:**
Store and analyze data on cloud platforms
- **Improved Machine Learning Models:**
Use advanced algorithms for higher prediction accuracy
- **Remote Monitoring:**
Enable full remote access using IoT platforms
- **Solar Power Integration:**
Use renewable energy for system operation
- **Real-Time Notifications:**
Implement SMS and mobile alerts

REFERENCES

17. REFERENCES

1. Buyya , R., & Dastjerdi , A. V. (2016). Internet of Things: Principles and Paradigms. Elsevier Publications.
2. Espressif Systems (2018). ESP8266 (NodeMCU) Technical Reference Manual.
3. Pedregosa, F., Varoquaux , G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
4. Kaur, G., & Kaur, J. (2017). Smart Irrigation System Using IoT. International Journal of Engineering Research & Technology (IJERT), 6(5), 1–5.
5. Kim, Y., Evans, R. G., & Iversen, W. M. (2008). Remote Sensing and Control of an Irrigation System Using a Distributed Wireless Sensor Network. IEEE Transactions on Instrumentation and Measurement, 57(7), 1379–1387.
6. Django Software Foundation (2023). Django Web Framework Documentation.
7. DHT11 Sensor Datasheet (Temperature and Humidity Sensor Specifications).
8. Soil Moisture Sensor Module Datasheet.
9. Ultrasonic Sensor (HC-SR04) Technical Documentation.
10. Sharma, V., & Kumar, R. (2020). IoT-Based Smart Agriculture Monitoring System. International Journal of Advanced Research in Computer Science, 11(3), 45–50.